



Politecnico
di Torino

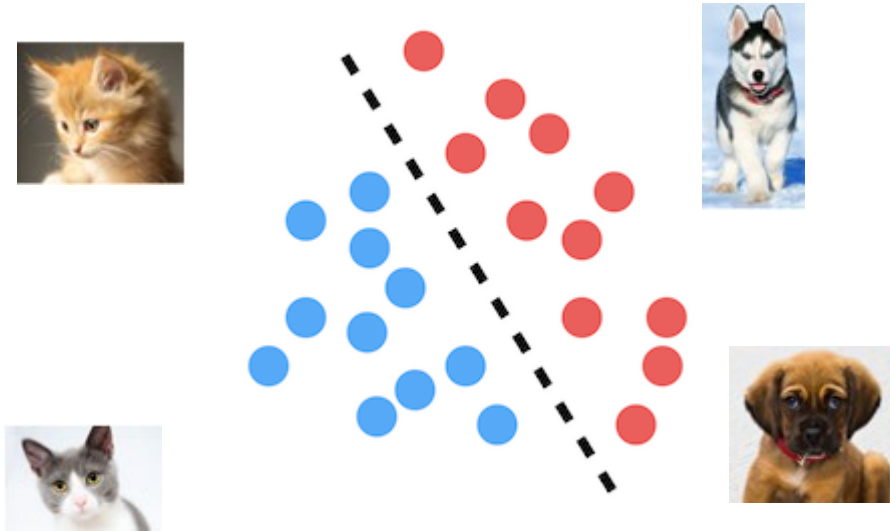
Generative Artificial Intelligence for graphics and multimedia

Generative adversarial networks

Prof. Lia Morra



Discriminative vs. Generative models

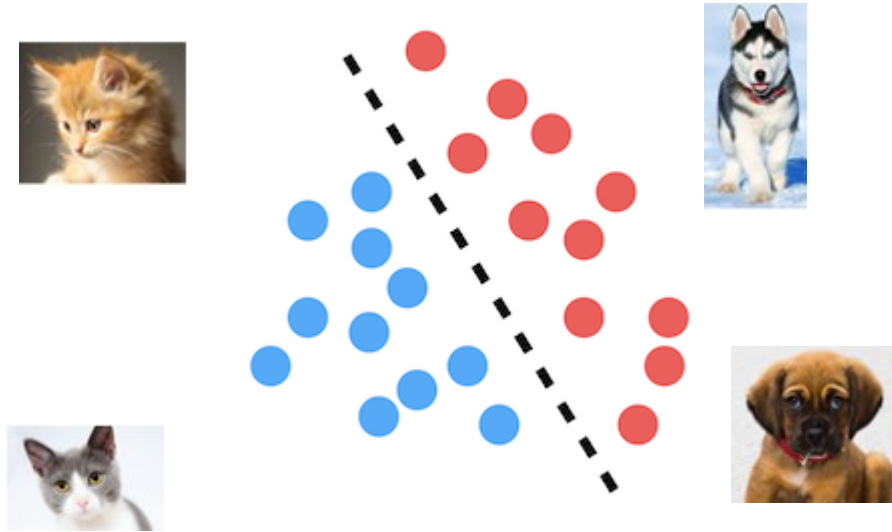


Features Class

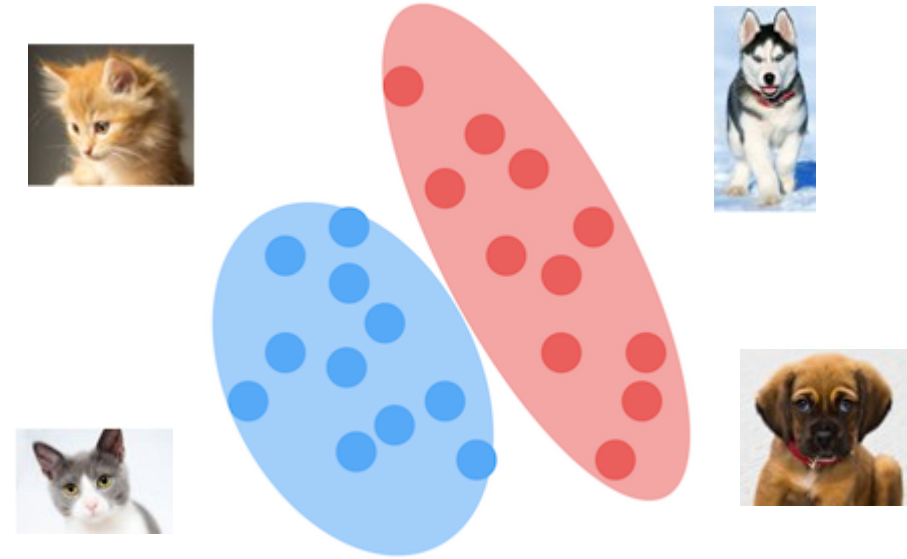
$x \rightarrow y$

$p(y | x)$

Discriminative vs. Generative models

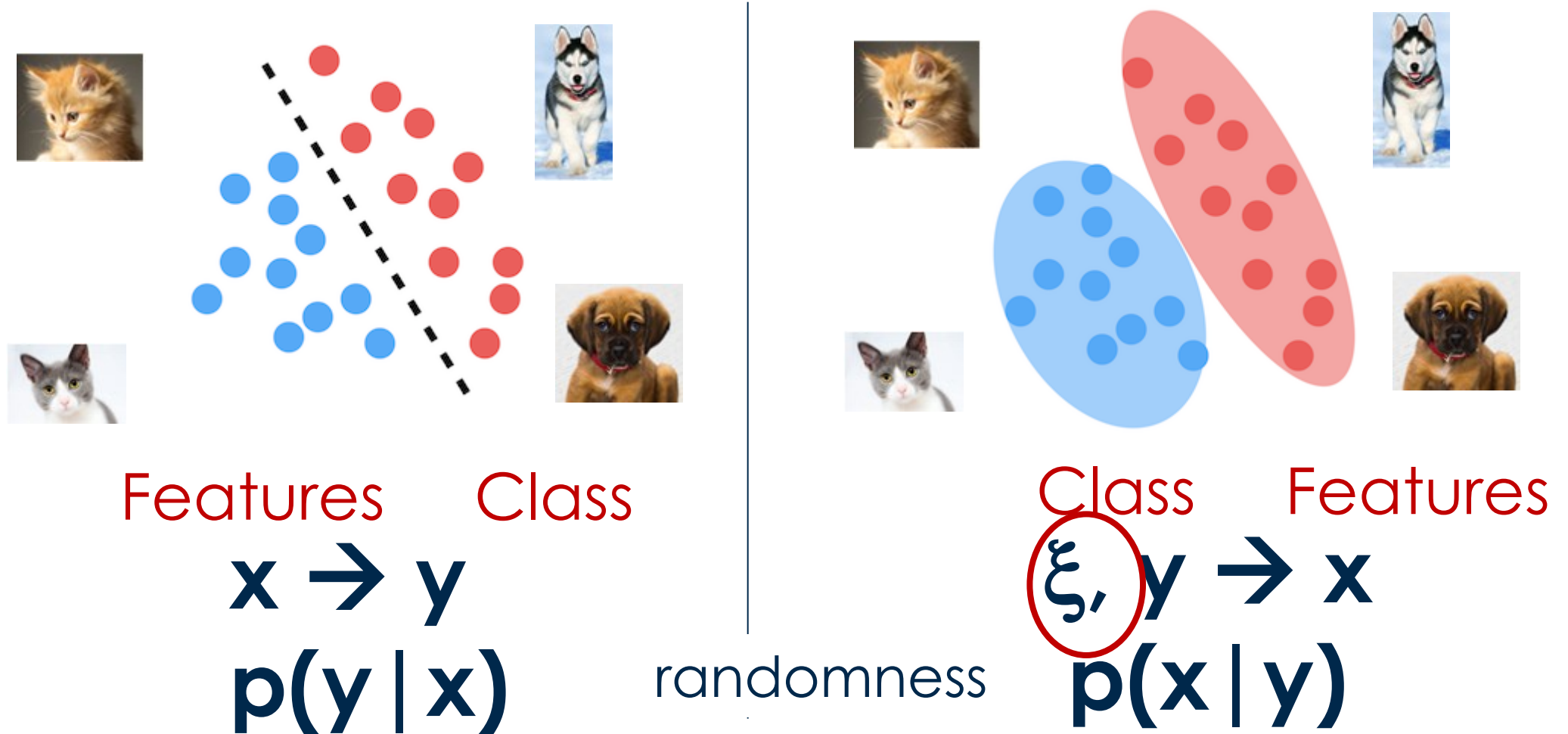


Features Class
 $x \rightarrow y$
 $p(y | x)$



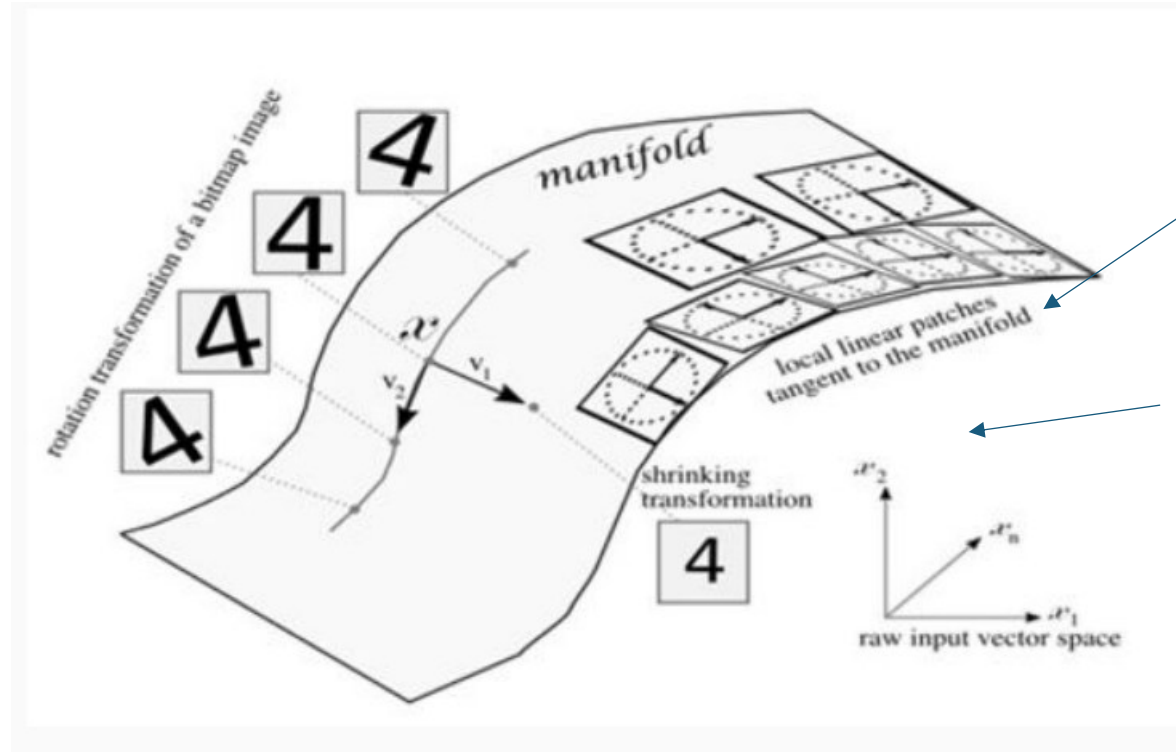
Class Features
 $\xi, y \rightarrow x$
 $p(x | y)$

Discriminative vs. Generative models



Manifold hypothesis

Natural (high-dimensional) data actually lies in a **low-dimensional** space



Unnatural data lies off the manifold
Unnatural data >> natural data

Goal: find a representation that i) **concentrates** only on natural images (data) and ii) captures **semantic similarity**

Source deeplearningbook.org

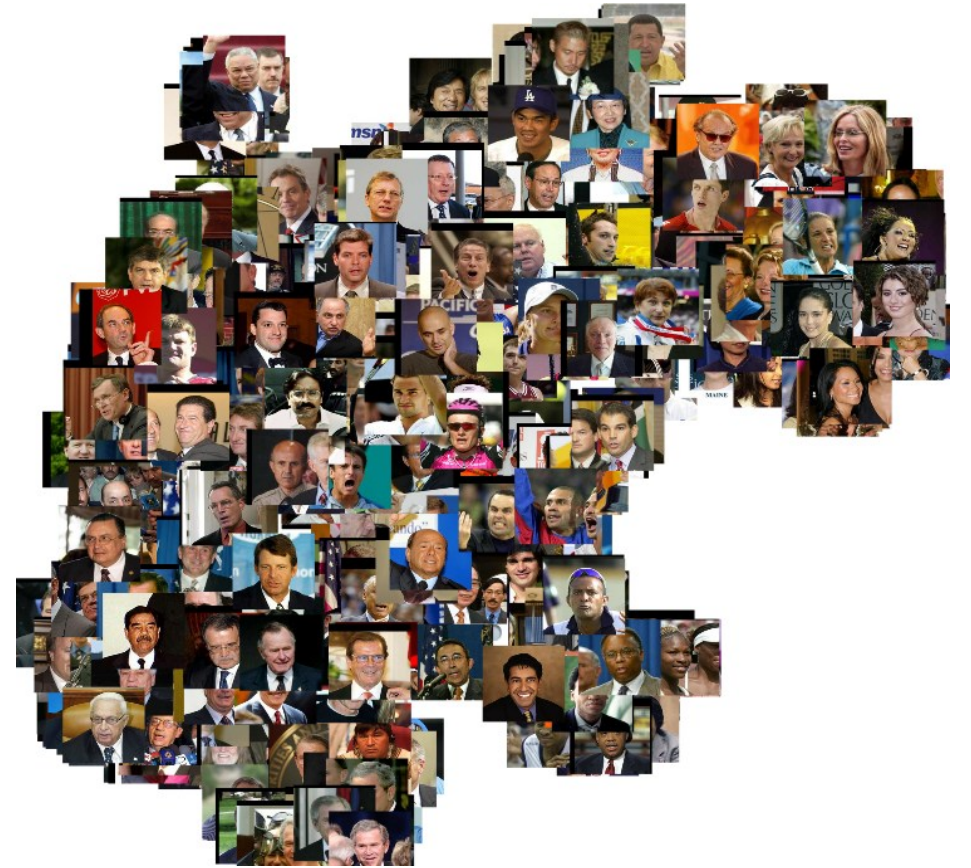
Latent space in generative models

The domain is a **manifold** in the original feature space

- › Manifold → subspace embedded in the original space
- › Example: the space of faces is a manifold of the space of the possible RGB images

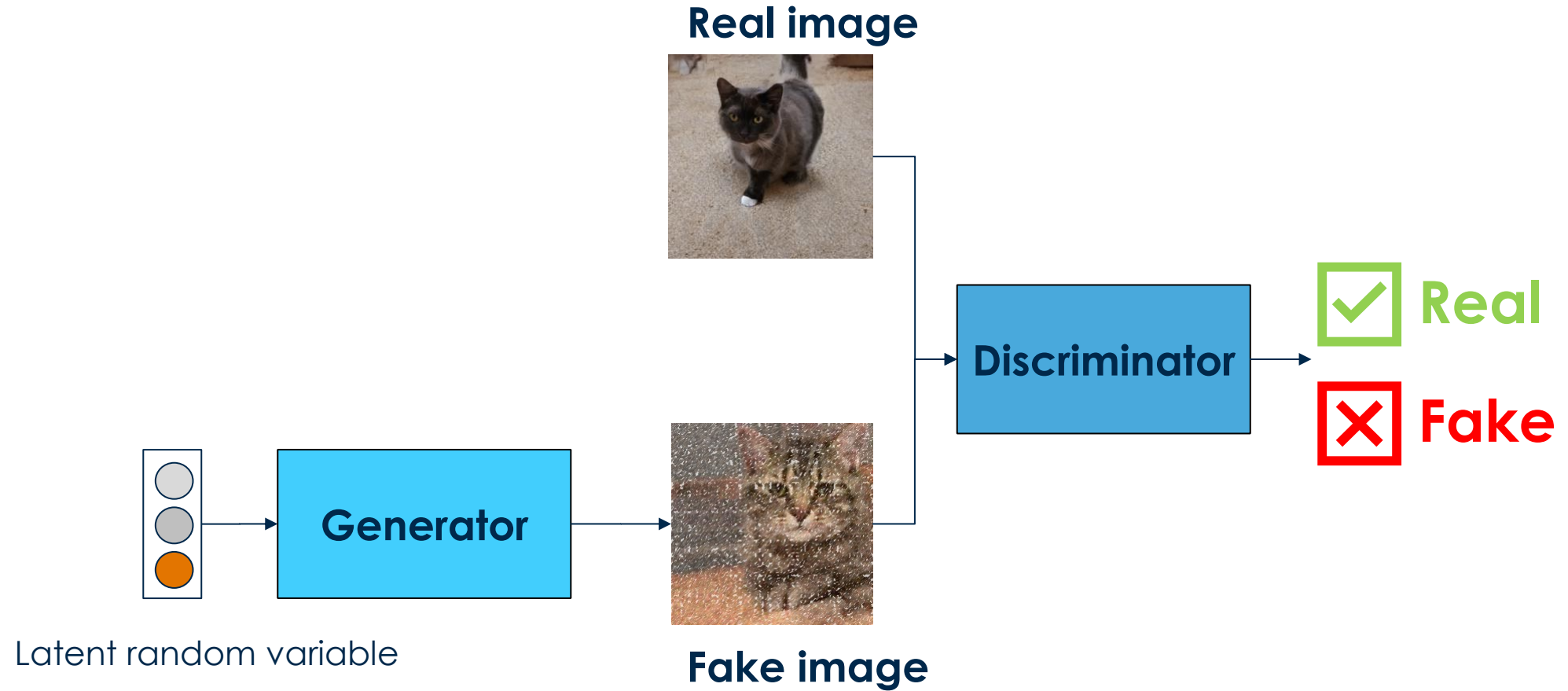
Manifolds have a **smaller intrinsic dimensionality** than the input space

- › Can be represented by a compact feature vector
- › These features can be complicated functions of the original input space

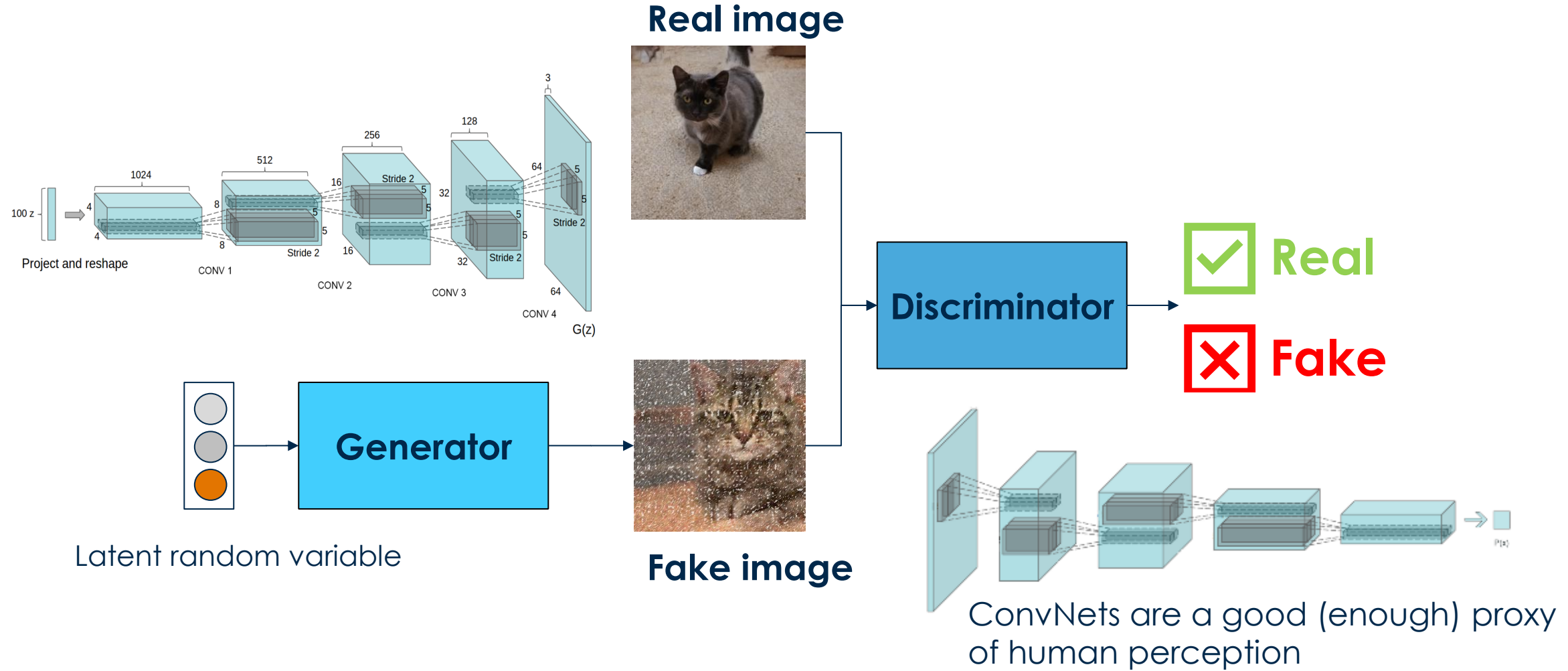


Sixue Gong, Vishnu Naresh Boddeti, Anil K. Jain, **On the Intrinsic Dimensionality of Image Representations**,
<https://arxiv.org/abs/1803.09672>

Generative Adversarial Network



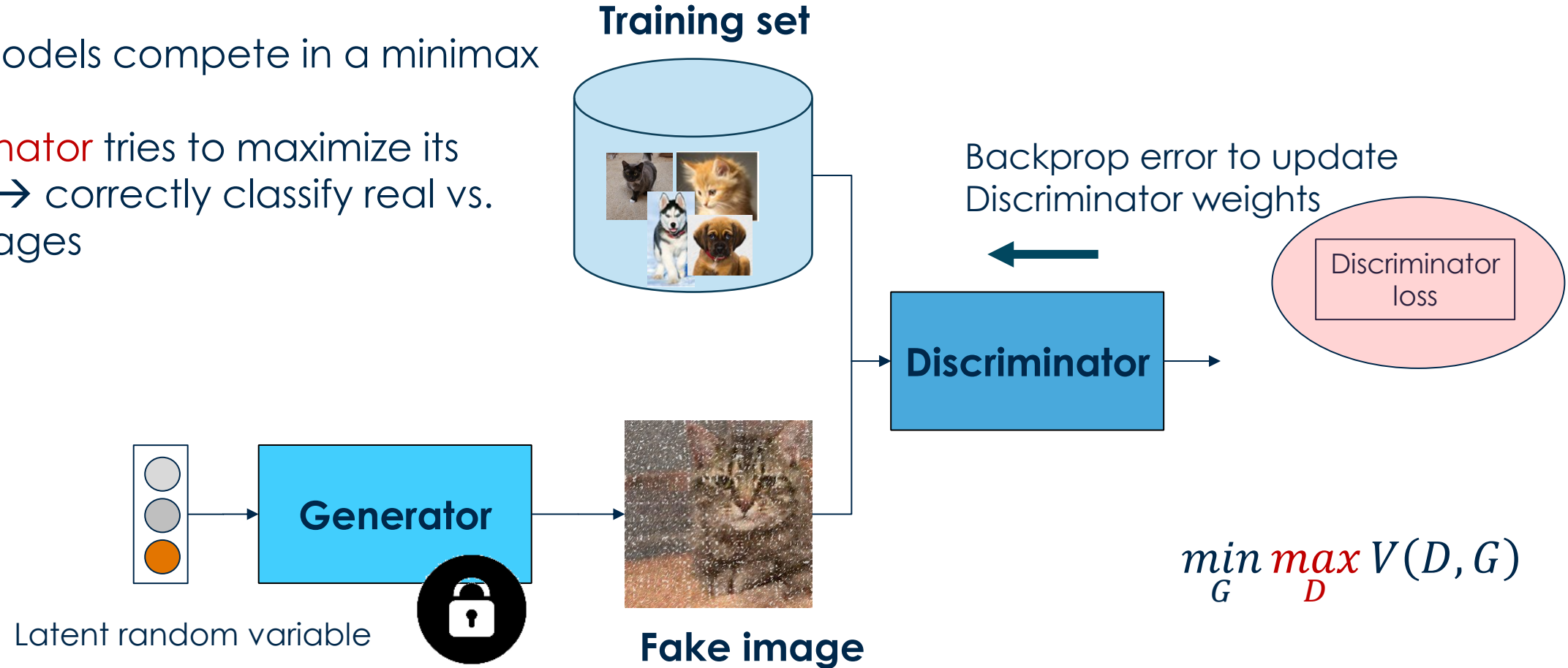
Generative Adversarial Network



How to train a GAN

The two models compete in a minimax game:

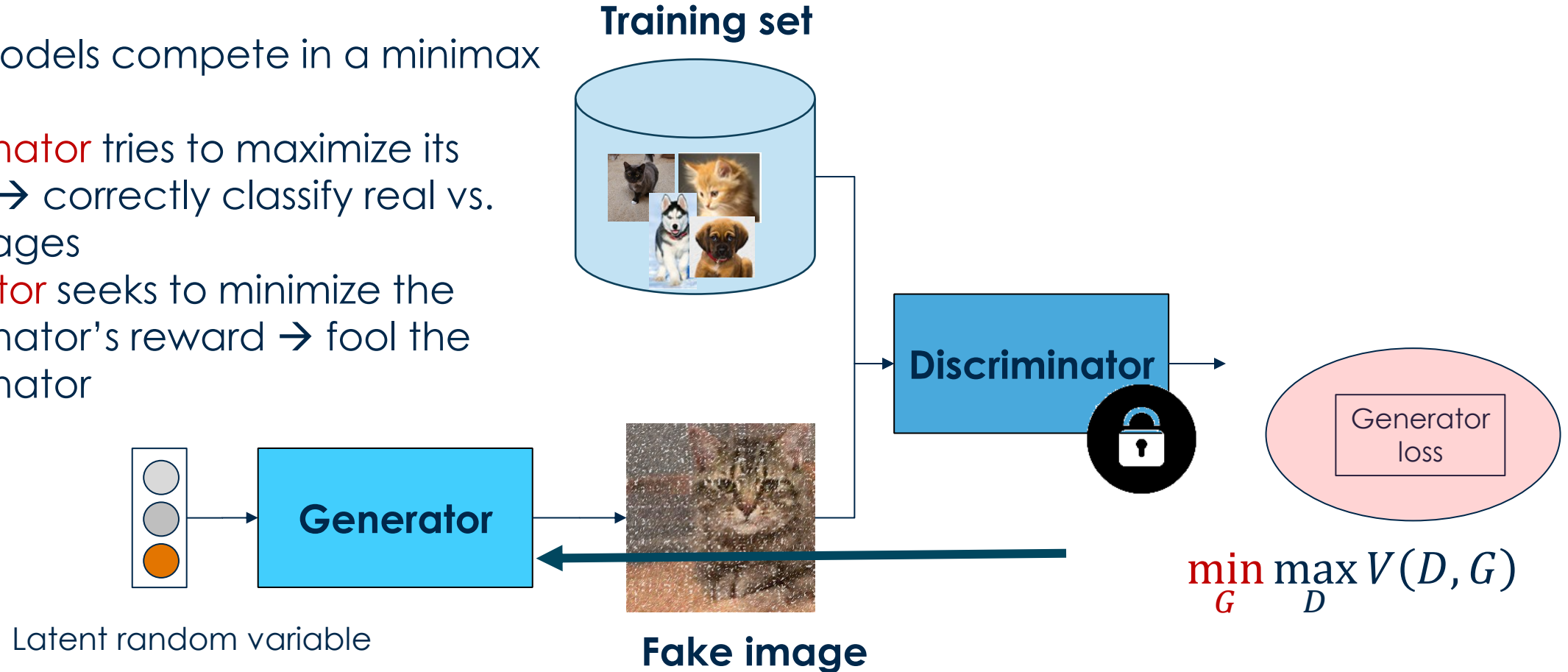
- **Discriminator** tries to maximize its reward → correctly classify real vs. fake images



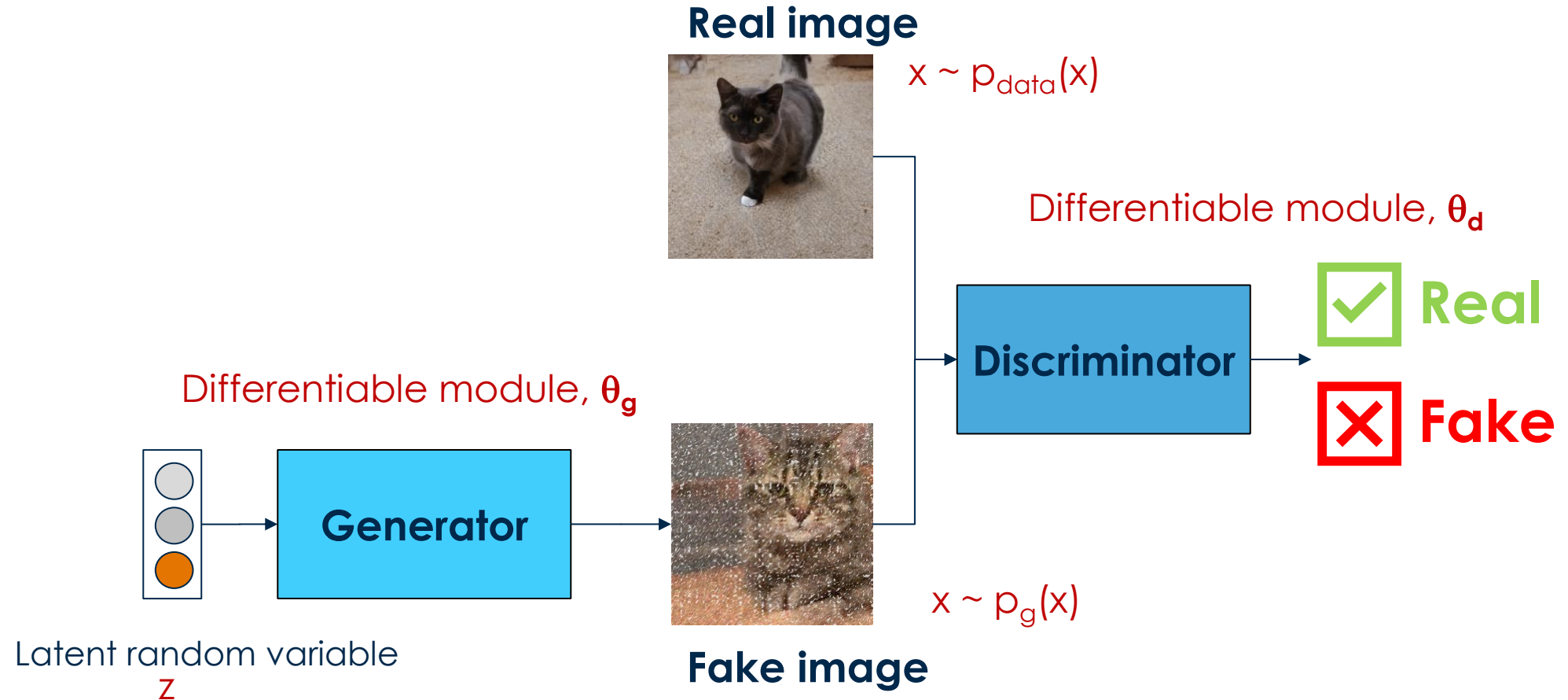
How to train a GAN

The two models compete in a minimax game:

- **Discriminator** tries to maximize its reward → correctly classify real vs. fake images
- **Generator** seeks to minimize the Discriminator's reward → fool the Discriminator



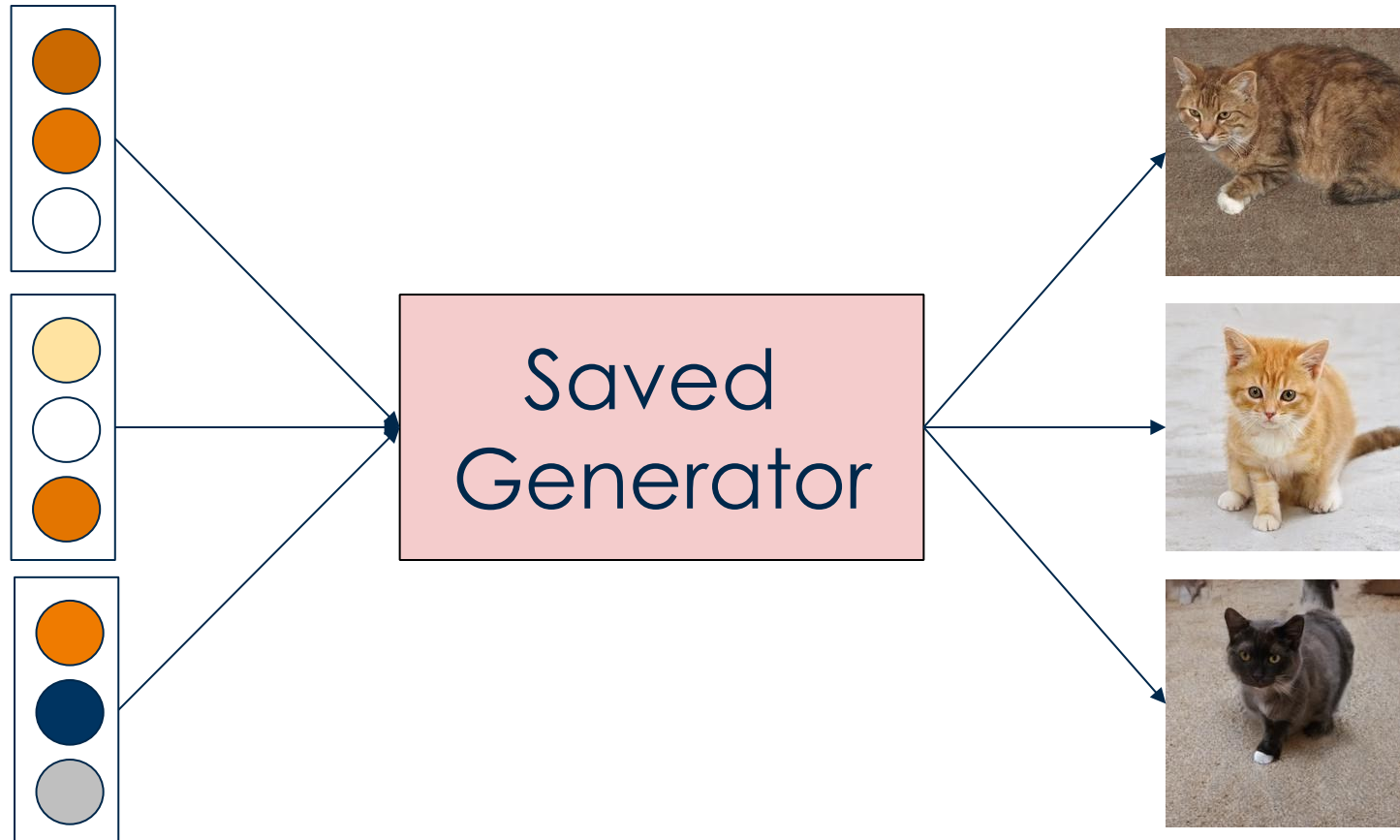
Notation



Sampling

The generator models $p(x | y)$

- › In case of a single domain (e.g., cats) it models $p(x)$

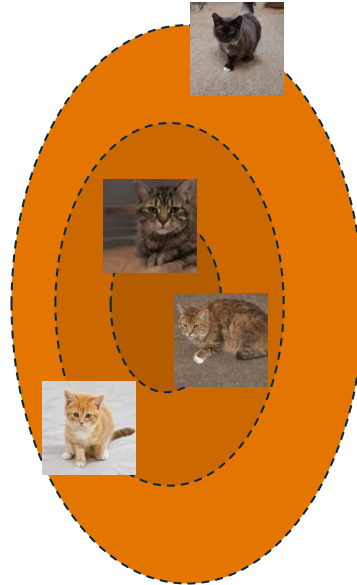


Which distribution?

The distribution approximation depends from the characteristics of the training set

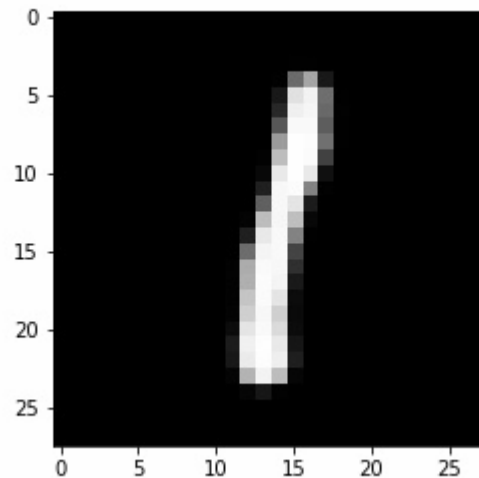
- › Most “common” samples in the training sets are more likely to be generated

$p(x)$

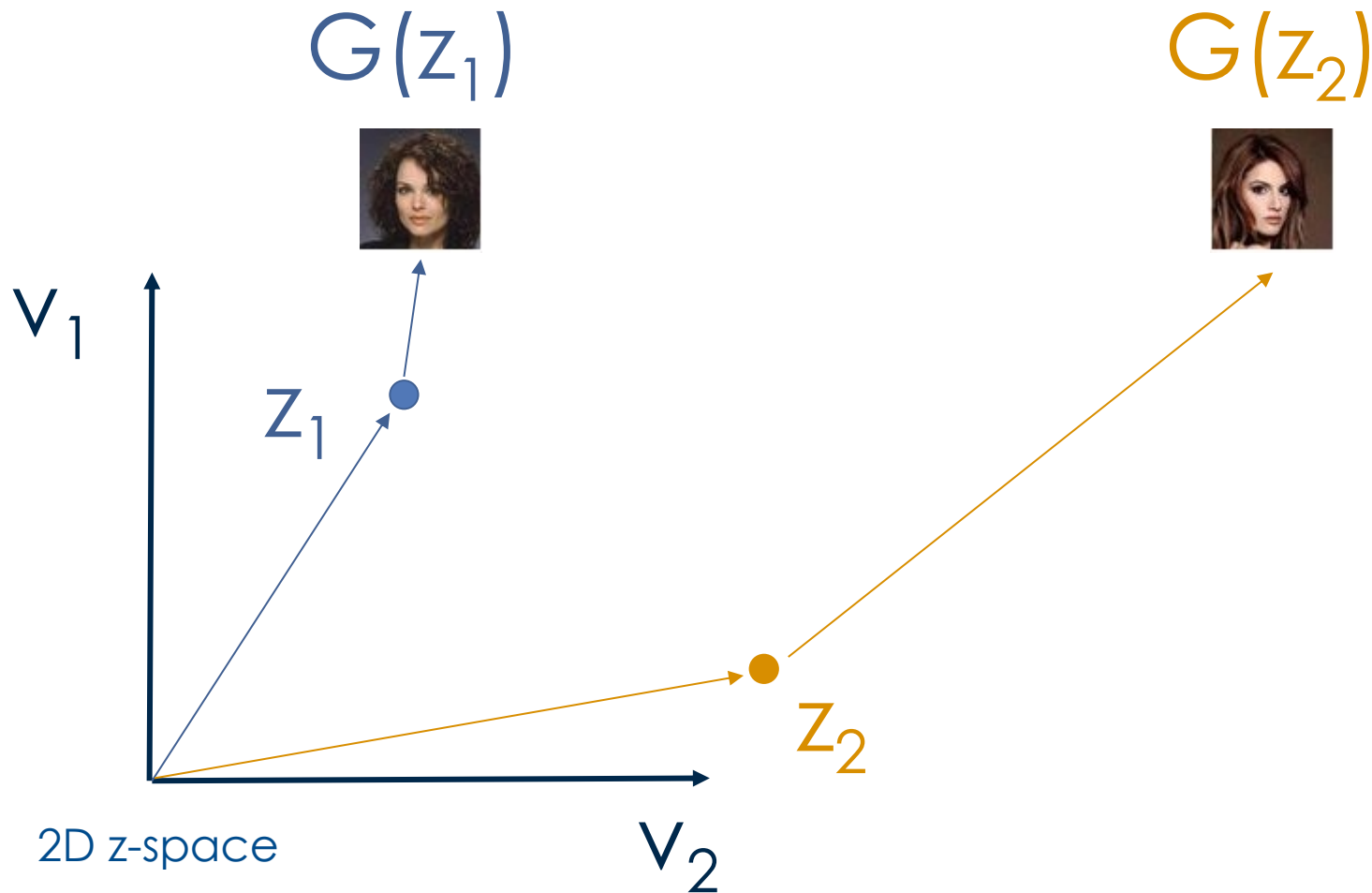


Latent space interpolation

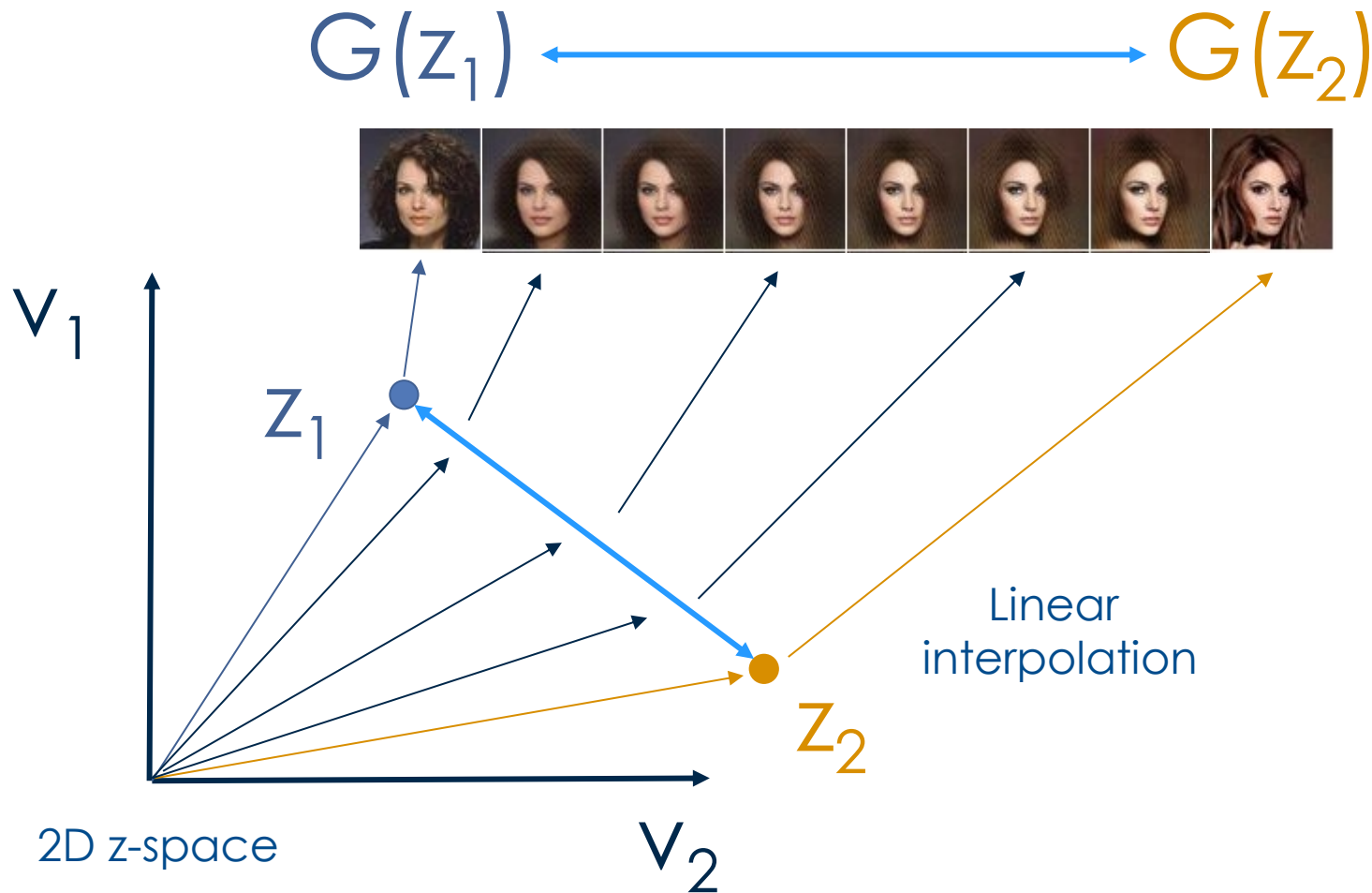
- Interpolating in latent space (z space $\rightarrow$ noise vector space) yields meaningful outputs
 - Compared to interpolating in pixel space!



Interpolation in the z-space



Interpolation in the z-space



A deeper look at GAN formulation

- Formulated as a minimax game where
 - › The discriminator is trying to maximize its reward
 - › The generator is trying to minimize the discriminator reward (or maximizing its loss)

$$\min_G \max_D V(D, G)$$

$$V(D, G) = \underbrace{\mathbb{E}_{x \sim p_{data}(x)} [\log D(x)]}_{\text{i.e. real } x \text{ is labelled real}} + \underbrace{\mathbb{E}_{z \sim p_g(z)} [\log(1 - D(G(z)))]}_{\text{i.e. fake } x \text{ is labelled fake}}$$

A deeper look at GAN formulation

Formulated as a **minimax** game where

- The discriminator is trying to maximize its reward
- The generator is trying to minimize the discriminator reward

$$\min_G \max_D V(D, G)$$

$$V(D, G) = \underbrace{\mathbb{E}_{x \sim p_{data}(x)} [\log D(x)]}_{\text{i.e. real } x \text{ is labelled real}} + \underbrace{\mathbb{E}_{z \sim p_g(z)} [\log(1 - D(G(z)))]}_{\text{i.e. fake } x \text{ is labelled fake}}$$

The (Nash) equilibrium of this game is achieved at

$$P_{data}(x) = P_g(x) \quad \forall x$$
$$D(x) = \frac{1}{2} \quad \forall x$$

How to train G and D?

Discriminator needs to predict 1 for real image, 0 for fake ones

The loss function for the discriminator is:

$$J^D = V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_g(z)} [\log(1 - D(G(z)))]$$

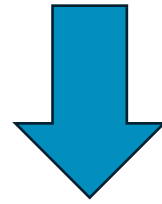
We want to **maximize** this loss function, so we perform **gradient ascent** as

$$\theta_d \leftarrow \theta_d + \mu \nabla V(\theta_d)$$

How to train G and D?

$$\min_G \max_D \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_g(z)} [\log(1 - D(G(z)))]$$

$$\text{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$



Fix G

$$\max_D \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_g(z)} [\log(1 - D(G(z)))]$$

$D(.) = 0$ «fake»
 $D(.) = 1$ «real»

This is a binary cross-entropy with a minus sign in front, but we are maxing → same loss as in a standard classifier

$D(.)$ encouraged to go to 1 when input is real data (sampled from p_{data})

$D(.)$ encouraged to go to 0 when input is fake data (generated by G)

How to train G and D?

Generator needs to fool the discriminator (it needs that the discriminator outputs 1 for fakes)

The loss function for the generator (no real images are used) is:

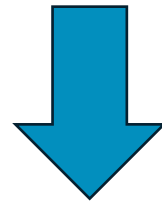
$$J^G = V(D, G) = \mathbb{E}_{z \sim p_g(z)} [\log(1 - D(G(z)))]$$

We want to **minimize** this loss function, so we perform **gradient descent** as

$$\theta g \leftarrow \theta g - \mu \nabla V(\theta g)$$

How to train G and D?

$$\min_G \max_D \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_g(z)} [\log(1 - D(G(z)))]$$



Fix D

$$\min_G \mathbb{E}_{z \sim p_g(z)} [\log(1 - D(G(z)))]$$

$D(.) = 0$ «fake»

$D(.) = 1$ «real»

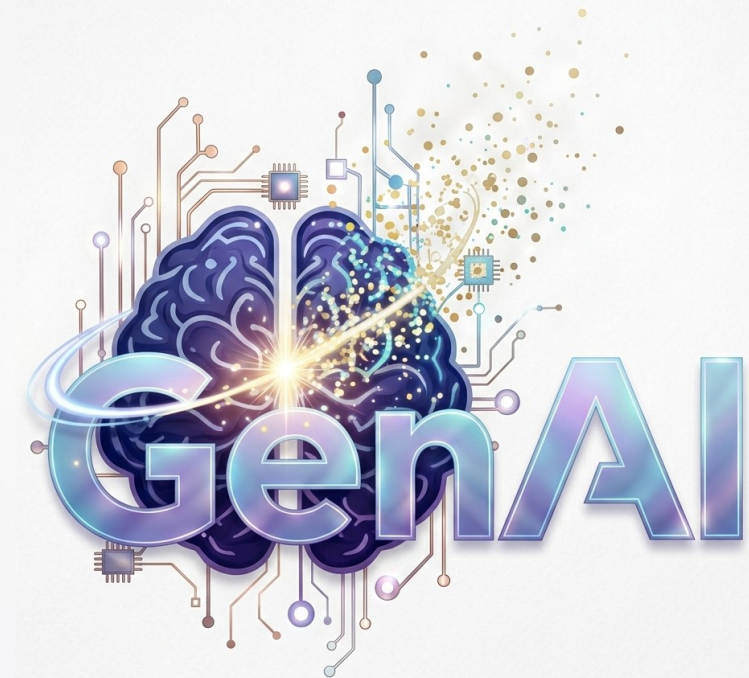


G is optimized so that it makes D output a value close to 1 («real», even though it is fake)



Politecnico
di Torino

GANs: Training challenges



Training a GAN

- Training is general unstable
- With alternate training of models, both models should improve together
 - G and D should be matched in capacity
 - If D is too good or too bad, training collapse
- Rule of thumb: D's accuracy should stay around 50% (guessing)

Issues with GAN formulation

When D is too confident, the gradient for G goes to zero ($\rightarrow$ vanishing gradient on the generator)

$$J^D = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_g(z)} [\log(1 - D(G(z)))]$$

$$J^G = \mathbb{E}_{z \sim p_g(z)} [\log(1 - D(G(z)))]$$

$$\nabla_{\theta_G} V(D, G) = \nabla_{\theta_G} \mathbb{E}_{z \sim p_g(z)} [\log(1 - D(G(z)))]$$

$$\nabla_a \log(1 - \sigma(a)) = \frac{-\nabla_a [1 - \sigma(a)]}{1 - \sigma(a)} = \frac{-\sigma(a) (1 - \sigma(a))}{1 - \sigma(a)} = -\sigma(a) = -D(G(z))$$

$$\nabla_{\theta_G} V(D, G) \propto -D(G(z)) \nabla_{\theta_G} G(z)$$

Alternative formulation

Alternative formulation → generator maximizes the log-probability of the discriminator being mistaken

$$J^D = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_g(z)} [\log(1 - D(G(z)))]$$

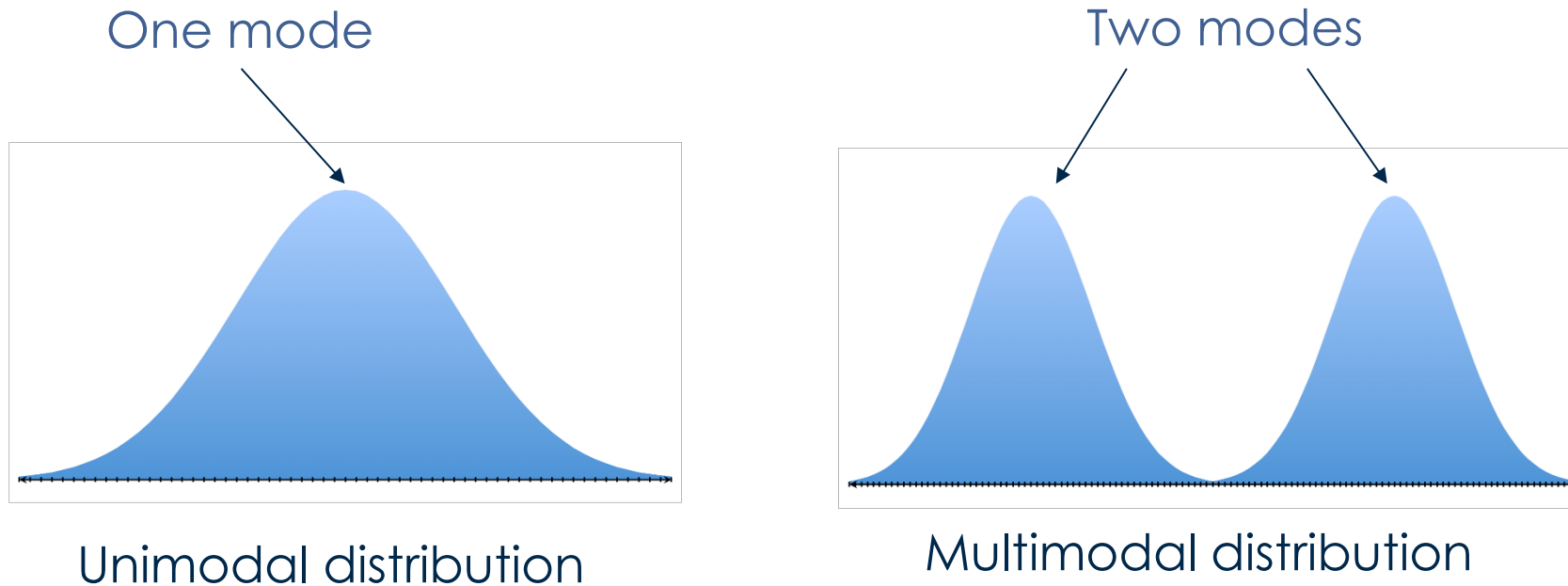
$$J^G = \mathbb{E}_{z \sim p_g(z)} [\log(D(G(z)))]$$


This is a binary cross-entropy with a minus sign in front (remember we are maxing J^G) with $y = 1$. Same loss as in a standard classifier.

The generator can still learn even when the discriminator successfully rejects (almost) all generator samples.

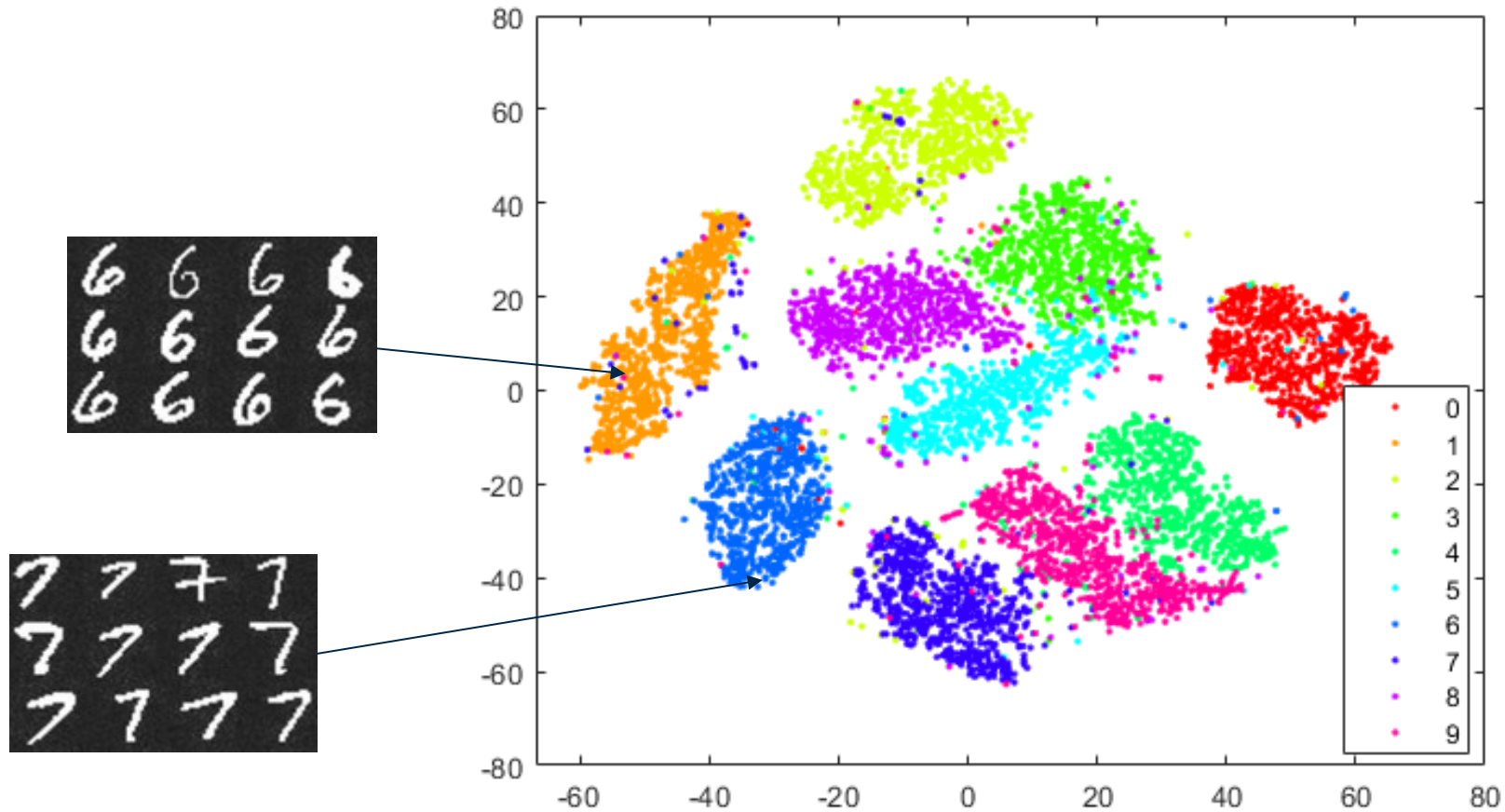
Issues with GANs: mode collapse

Real data distributions are multi-modal

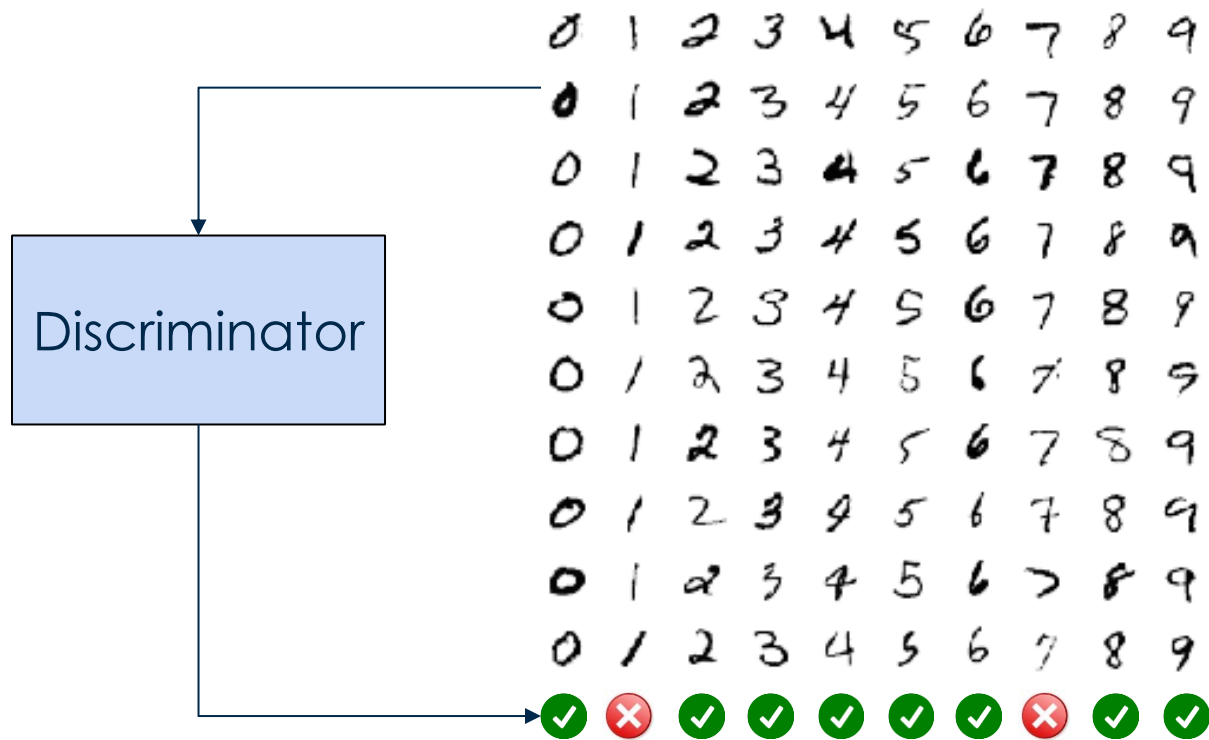


Intuitively, any peak in the feature distribution is a mode

Issues with GANs: mode collapse



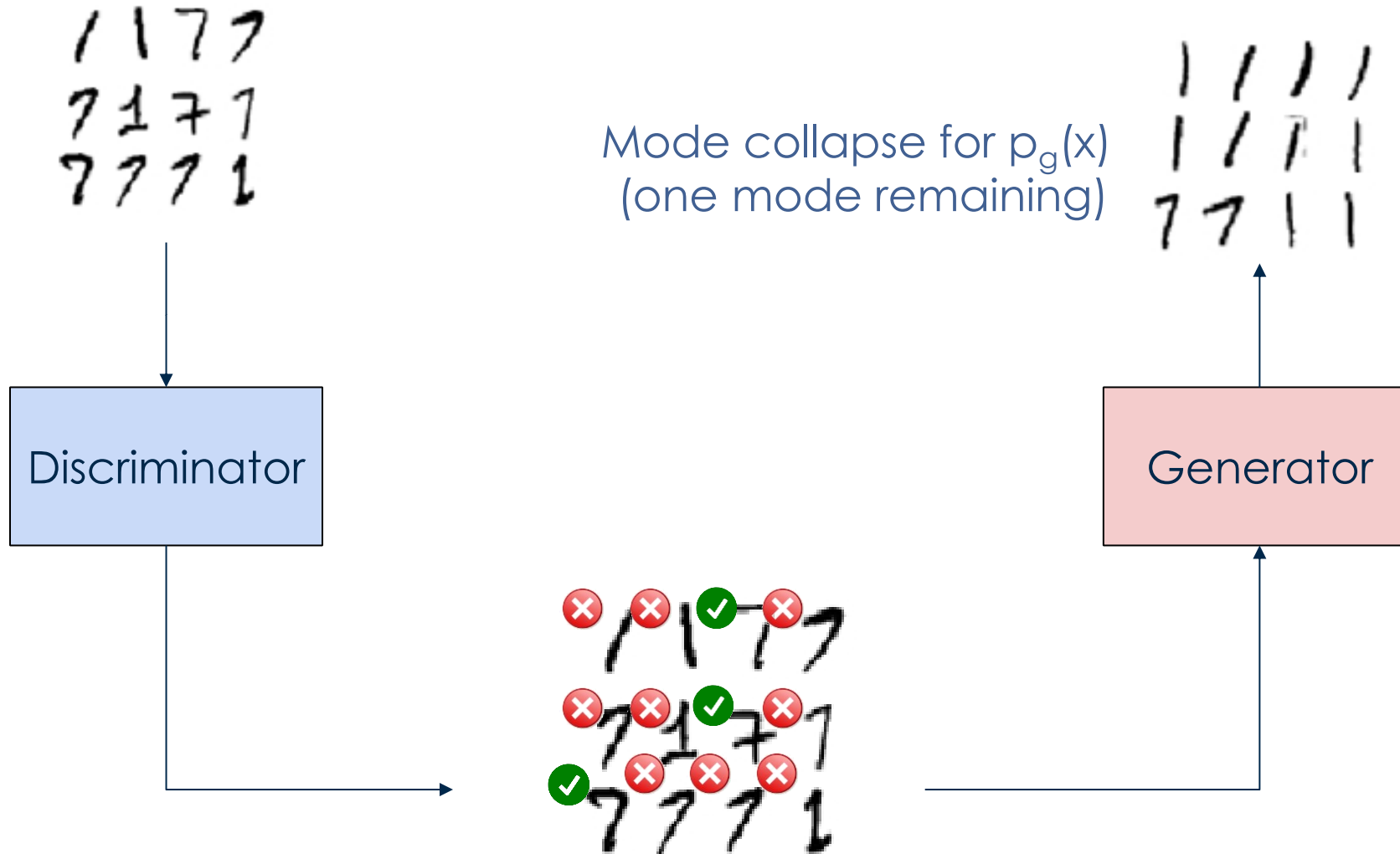
Mode collapse



Mode collapse



Mode collapse



Alternative formulation: non-saturating loss

Alternative formulation → generator maximizes the log-probability of the discriminator being mistaken

$$J^D = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_g(z)} [\log(1 - D(G(z)))]$$

$$J^G = \mathbb{E}_{z \sim p_g(z)} [\log(D(G(z)))]$$

Using the $-\log D$ trick, it can be shown that the resulting loss has high cost for fake looking samples but low cost for missing modes (**mode dropping**)

"Towards Principled Methods for Training Generative Adversarial Networks" – Arjovsky et al., ICLR 2017

Wasserstein GAN (WGAN)

- State-of-the-art GANs do not use the original formulation
 - variants include least-square loss, hinge loss, Wasserstein GAN
- Wasserstein GAN is based on the concept of **Wasserstein** or **Earth mover distance** between the real and fake distribution

Cost of the optimal transport

$$W(p_{data}, p_g) = \inf_{\gamma \sim \Pi(p_{data}, p_g)} E_{(x,y) \sim \gamma} [|x - y|]$$

$$\begin{aligned} \Pi(p_{data}, p_g) &= \{ \gamma(x, y) \mid \int \gamma(x, y), dy = p_{data}(x), \quad \int \gamma(x, y), dx = p_g(y) \} \end{aligned}$$

Joint distribution



Source: <https://www.microsoft.com/en-us/research/blog/measuring-dataset-similarity-using-optimal-transport/>

Arjovsky et al. «Wasserstein Generative Adversarial Networks», ICML 2017

Wasserstein GAN (WGAN)

The Wasserstein distance admits a dual formulation (Kantorovich-Rubinstein) – which is tractable

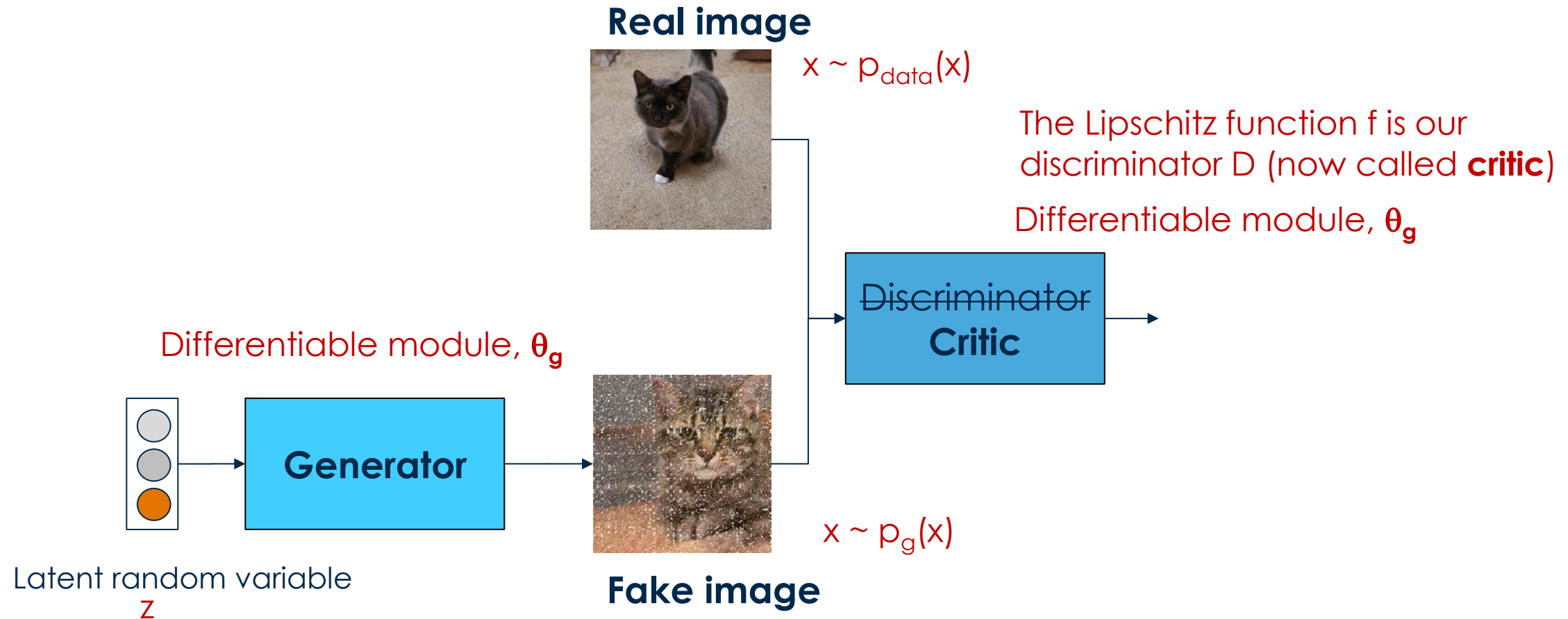
$$W(p_{data}, p_g) = \sup_{|f|_L \leq 1} E_{x \sim p_{data}}[f(x)] - E_{x \sim p_g}[f(x)]$$

where the supremum is over all **Lipschitz functions** f , meaning:

$$|f(x) - f(y)| \leq |x - y| \quad \forall x, y$$

Intuition: the function f cannot “grow too fast”

The critic in Wasserstein GAN



Training the critic with gradient penalty (WGAN-GP)

From the dual formulation we derive the loss of the critic:

$$\mathcal{L}_{\mathcal{D}} = -E_{x \sim p_{data}}[D(x)] + E_{z \sim p_z(z)}[D(G(z))]$$

Problem: the critic must be a Lipschitz function

- simple solution: **clip the weights** between -1 and 1
- more elegant solution: Lipschitz constraint is enforced by promoting **critic gradients to be small** in the loss function (penalty)

$$\mathcal{L}_{\mathcal{D}} = -E_{x \sim p_{data}}[D(x)] + E_{z \sim p_z(z)}[D(G(z))] + \lambda E_{\hat{x} \sim p_{\hat{x}}}[(|\nabla_{\hat{x}} D(\hat{x})|_2 - 1)^2]$$


$$\hat{x} = \epsilon x + (1 - \epsilon)G(z), \quad \epsilon \sim \mathcal{U}[0,1]$$

Random combination of a real and a fake sample

Training the generator

The generator wants to **minimize** the Wasserstein distance or, conversely, to **maximize** the critic's output on fake samples, i.e. maximize $\mathbb{E}[D(G(z))]$, which is equivalent to minimizing the negative:

$$\mathcal{L}_G = -\mathbb{E}_{z \sim p_z(z)}[D(G(z))]$$

This is fundamentally different from the original GAN — the critic D is **not** a classifier outputting probabilities and the gradient signal remains **non-saturating** throughout training.

Sketching the training loop – training the critic

```
for epoch in range(EPOCHS):
```

```
    # Sample real data
```

```
    for i, (real_images, labels) in enumerate(data_loader):
```

```
        .....
```

```
        # Sample noise as generator input
```

```
        z = torch.randn(BATCH_SIZE, LATENT_DIM, device=DEVICE)
```

```
        # Sample fake images
```

```
        fake_images = generator(z)
```

```
        # Compute the output of the critic
```

```
        real_out = critic(real_images)
```

```
        fake_out = critic(fake_images)
```

```
        gradient_penalty = compute_gradient_penalty(..)
```

```
        d_loss = torch.mean(fake_out) - torch.mean(real_out) + lambda * gradient_penalty
```

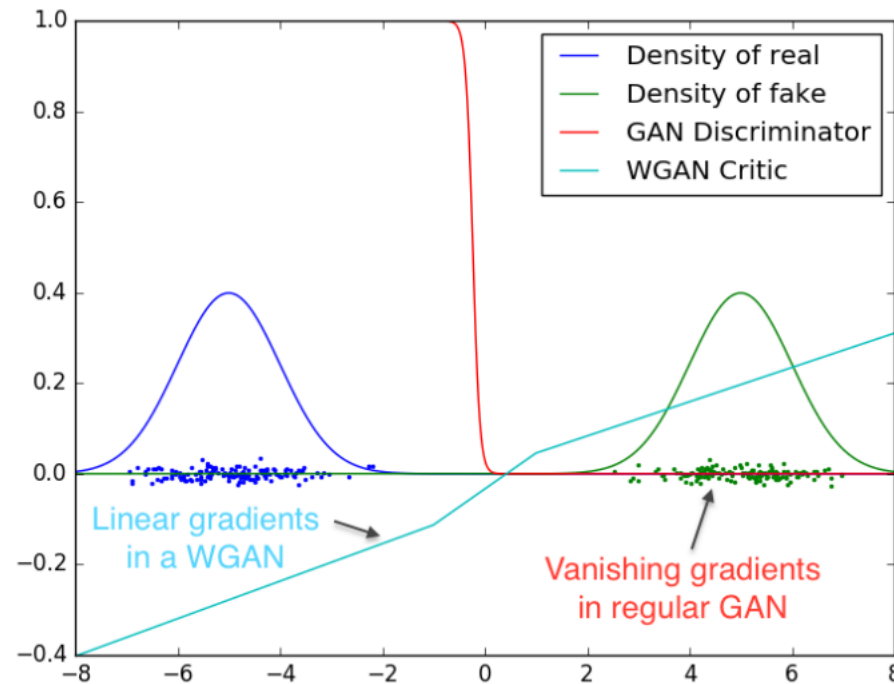
$$\begin{aligned}\mathcal{L}_{\mathcal{D}} &= -E_{x \sim p_{data}}[D(x)] + E_{z \sim p_z(z)}[D(G(z))] \\ &\quad + \lambda E_{\hat{x} \sim p_{\hat{x}}}[(|\nabla_{\hat{x}} D(\hat{x})|_2 - 1)^2]\end{aligned}$$

Implementing gradient penalty

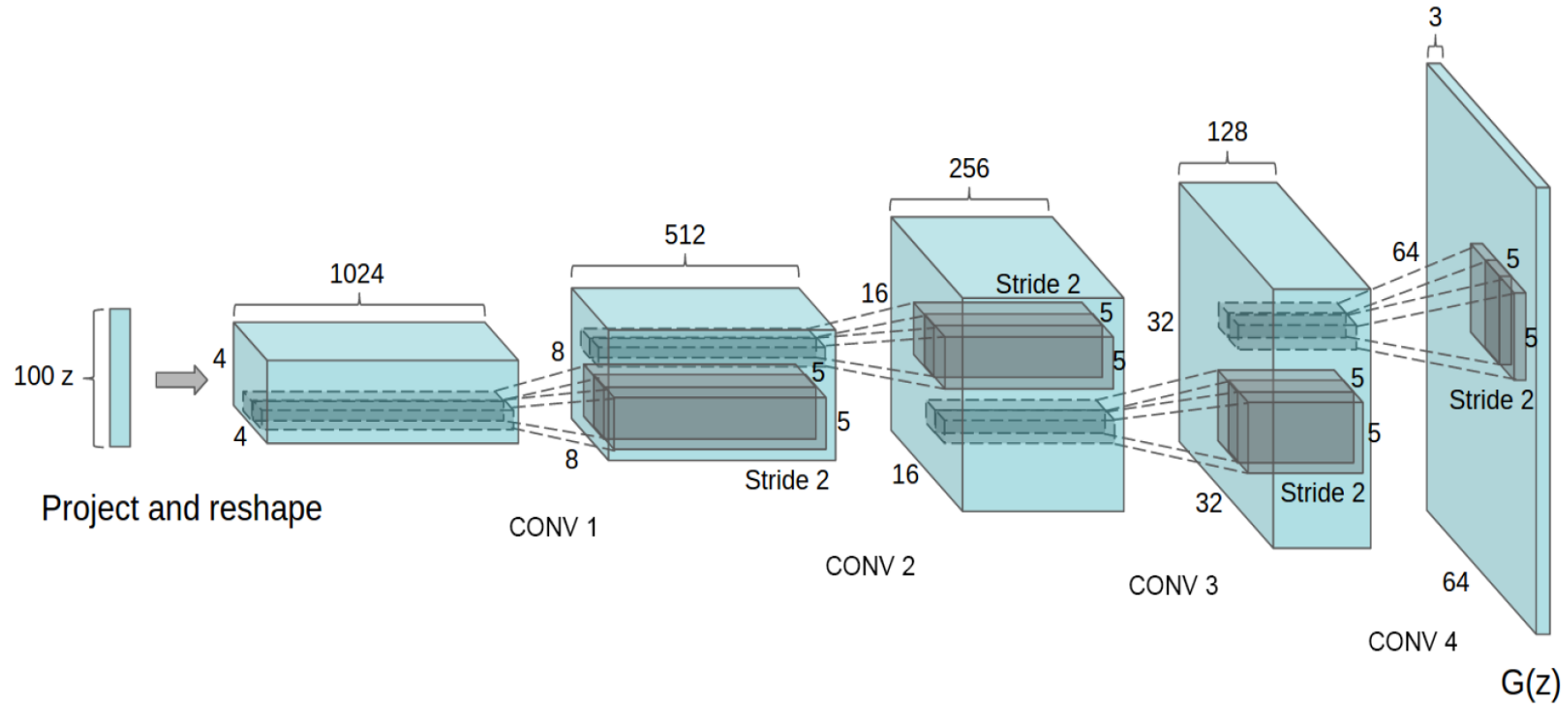
```
def compute_gradient_penalty(critic, real_samples, fake_samples):  
    # Random weight term for interpolation between real and fake samples  
    alpha = torch.rand(BATCH_SIZE, 1, 1, 1).to(DEVICE)  
    # Get random interpolation between real and fake samples  
    interpolated = ....  
    output = critic (interpolated)  
    # Get gradient w.r.t. interpolates  
    gradients = torch.autograd.grad(  
        outputs=logits, inputs=interpolated, grad_outputs=torch.ones_like(logits), create_graph=True  
    )[0]  
    gradients = gradients.view(BATCH_SIZE, -1)  
    # Compute the norm  
    gradient_penalty = ((gradients.norm(2, dim=1) - 1) ** 2).mean()  
    return gradient_penalty
```

Wasserstein GAN

Well-defined and non-vanishing gradients even when the distributions are supported on compact and disjoint manifolds



DCGAN architecture



Training DCGANs

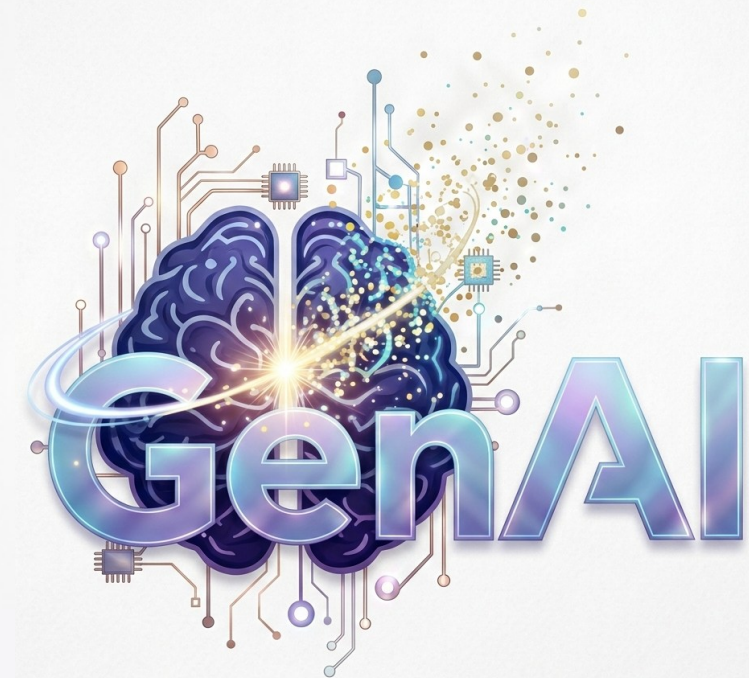
The breakthrough of the paper was putting forth the main key ideas/recommendations used in all following works

- › Replace max pooling with convolutional stride
- › Replace FC layers with conv layers
- › Use deconvolution (transposed convolutions) in generator for upsampling
- › Use batch normalization after each layer (except the generator output)
- › In the generator, use ReLU in hidden layers and tanh for the output
- › In the discriminator, use LeakyReLU



Politecnico
di Torino

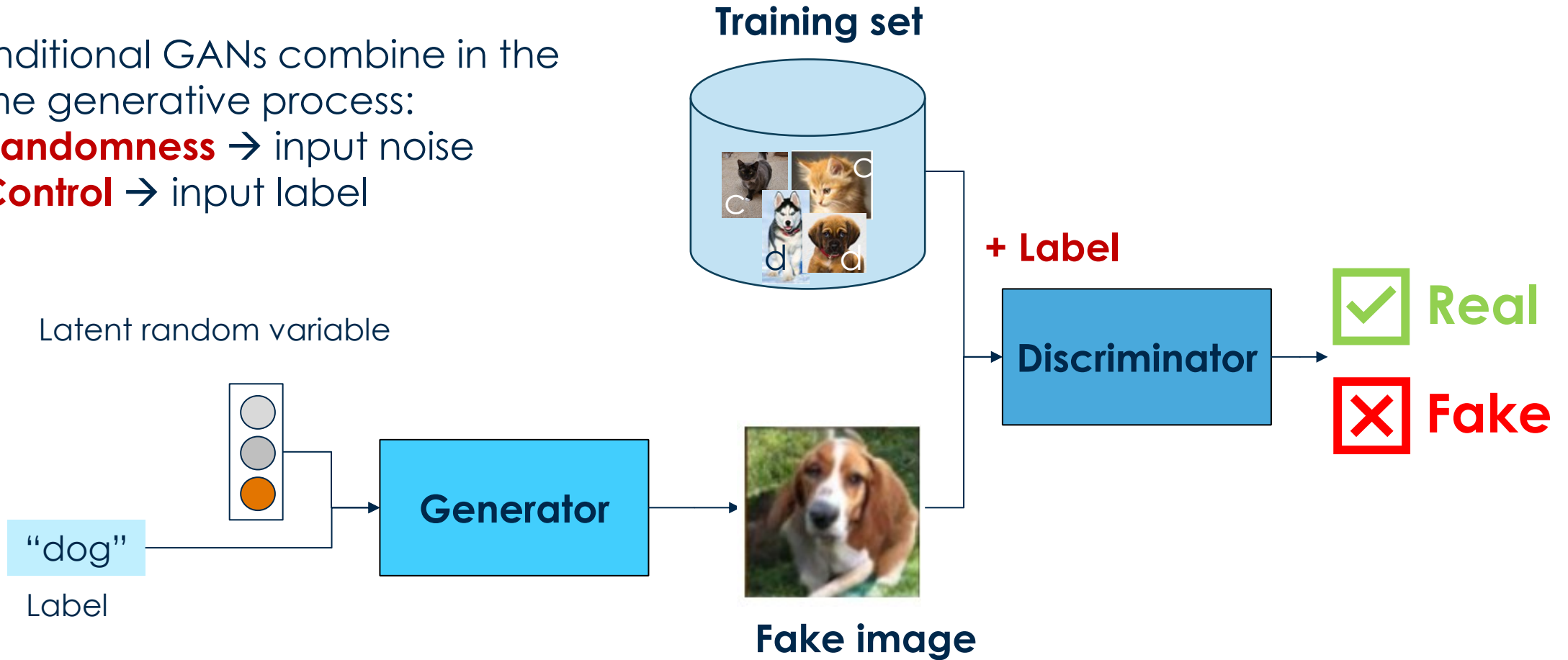
Conditional generation



Conditional GAN

Conditional GANs combine in the same generative process:

- **Randomness** → input noise
- **Control** → input label

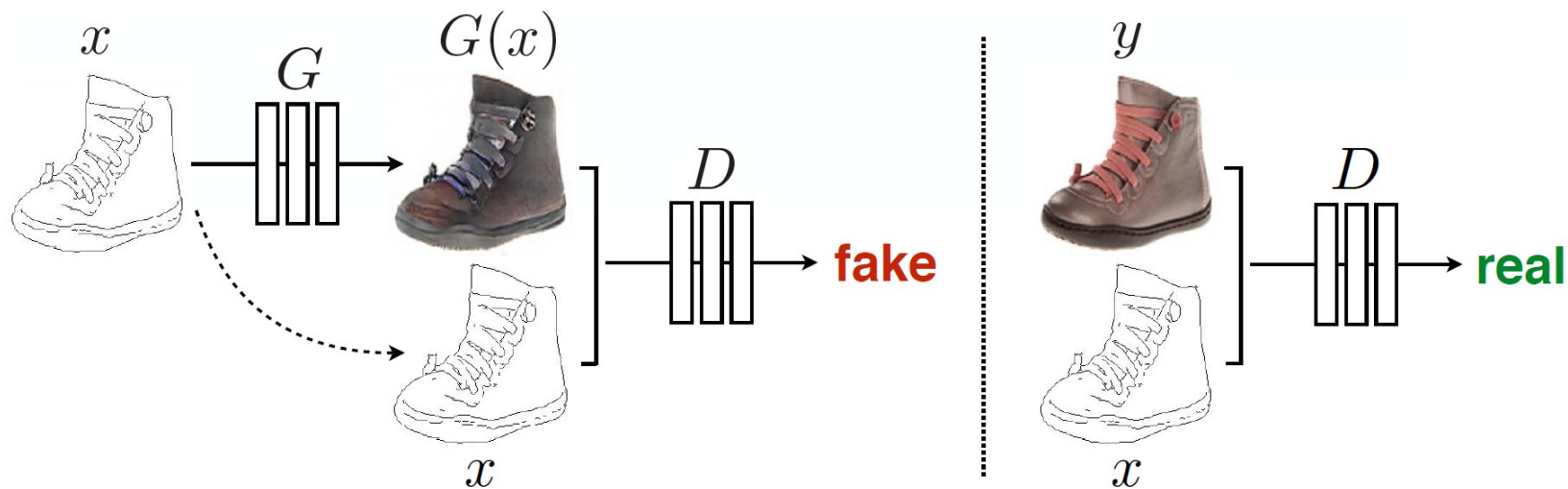


Pix2Pix

Generator takes an **image** as input (instead of a label)

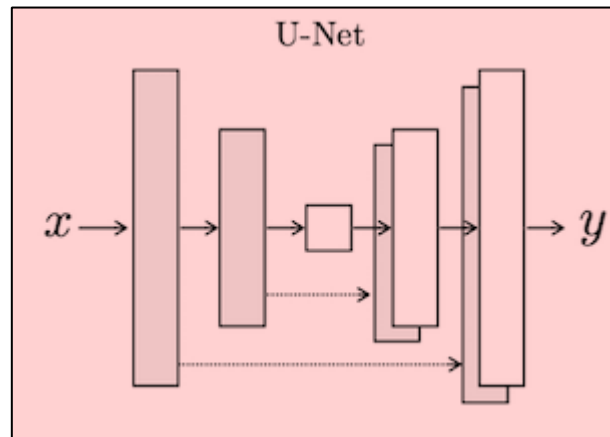
Generator aims at producing a specific, consistent output

Trained on **paired** image sets



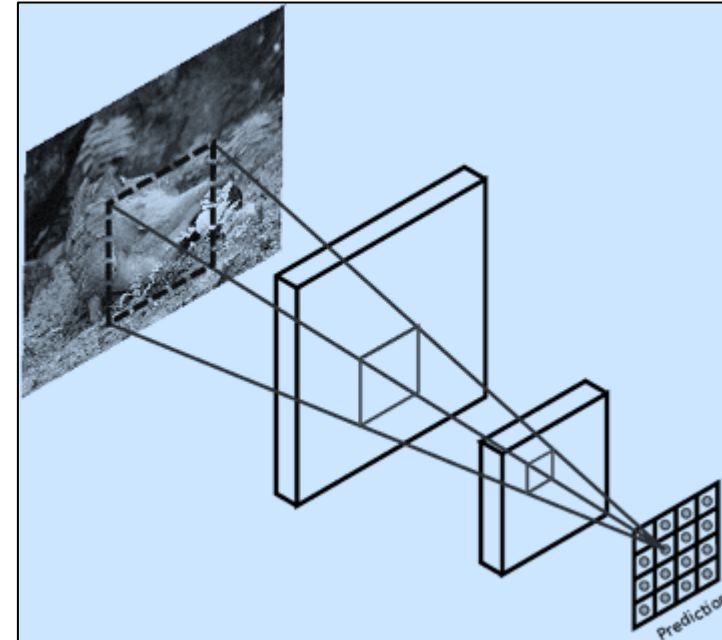
Pix2Pix module architecture

Pix2Pix
Generator



Autoencoder with
skip connections

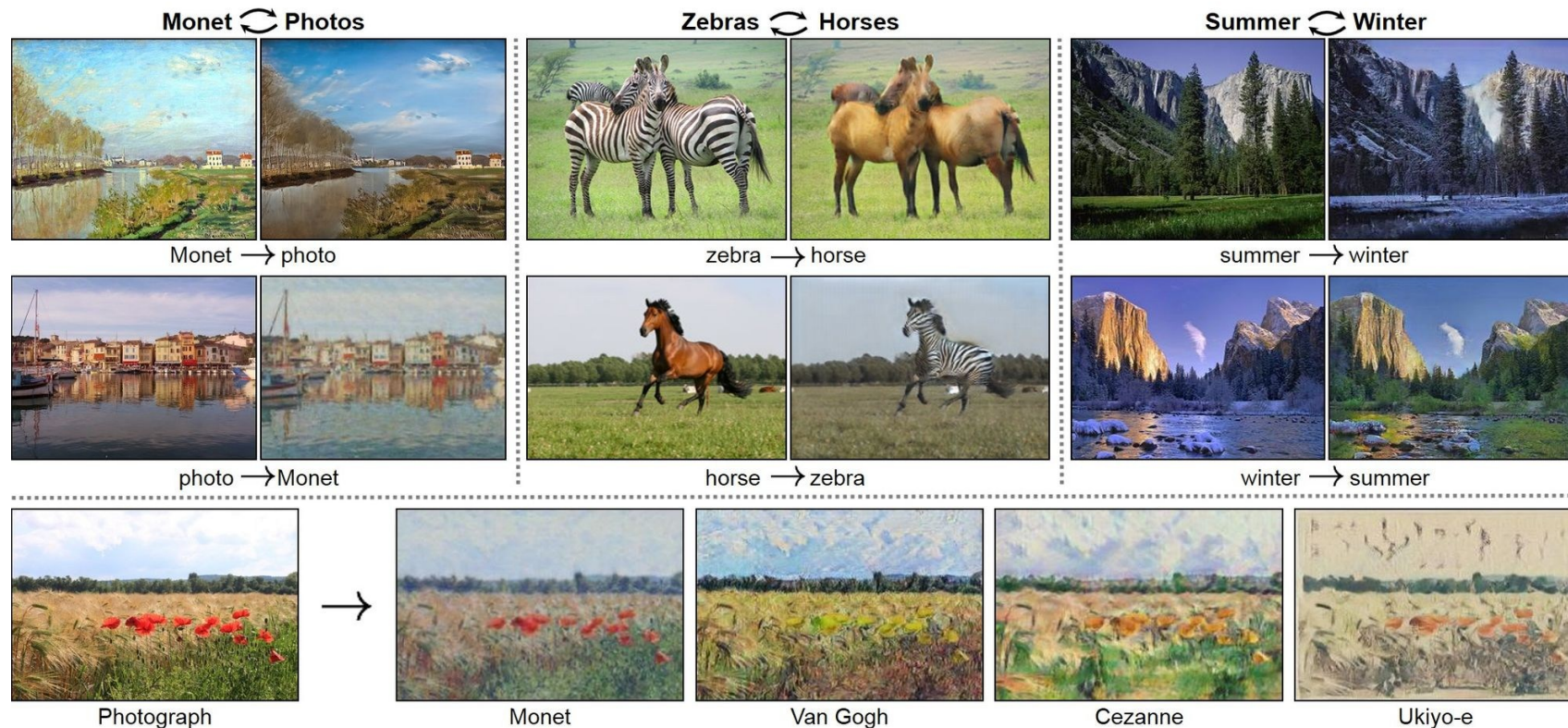
Pix2Pix
Discriminator



PatchGAN (decision at patch level)

CycleGAN

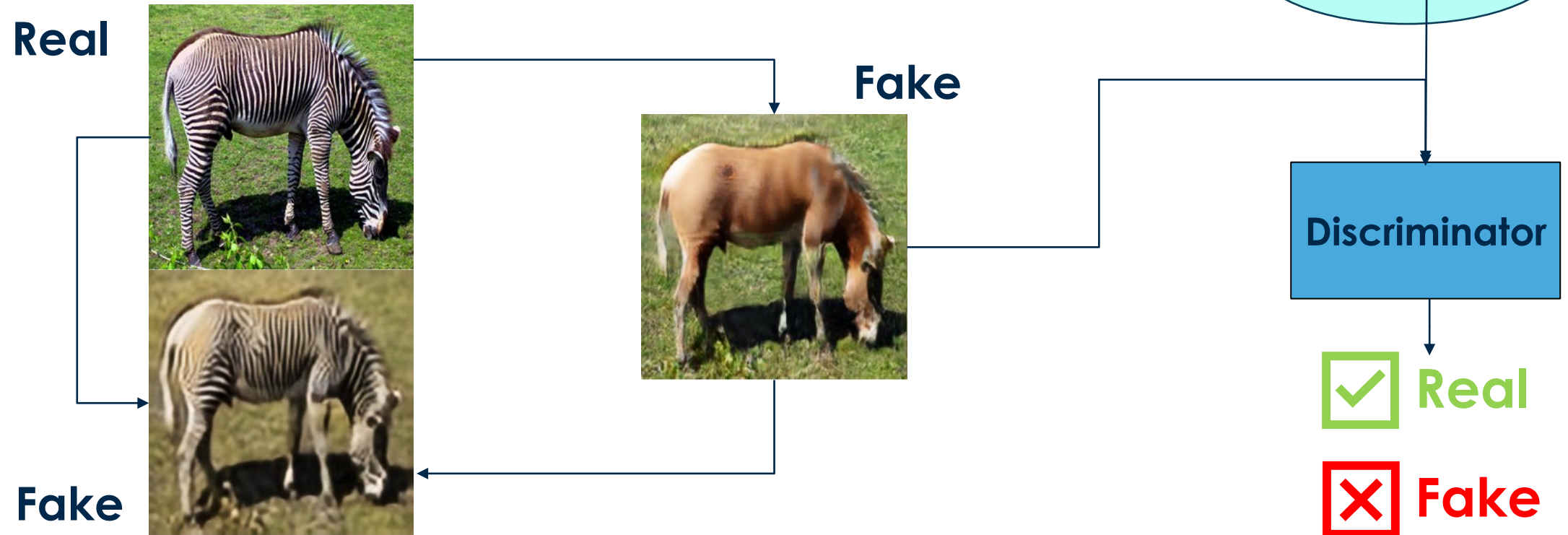
Find reciprocal mapping between images with similar content, but different style, from **unpaired** image sets



Cycle consistency

From zebra to horse and back to zebra ($\rightarrow$ **cycle**)

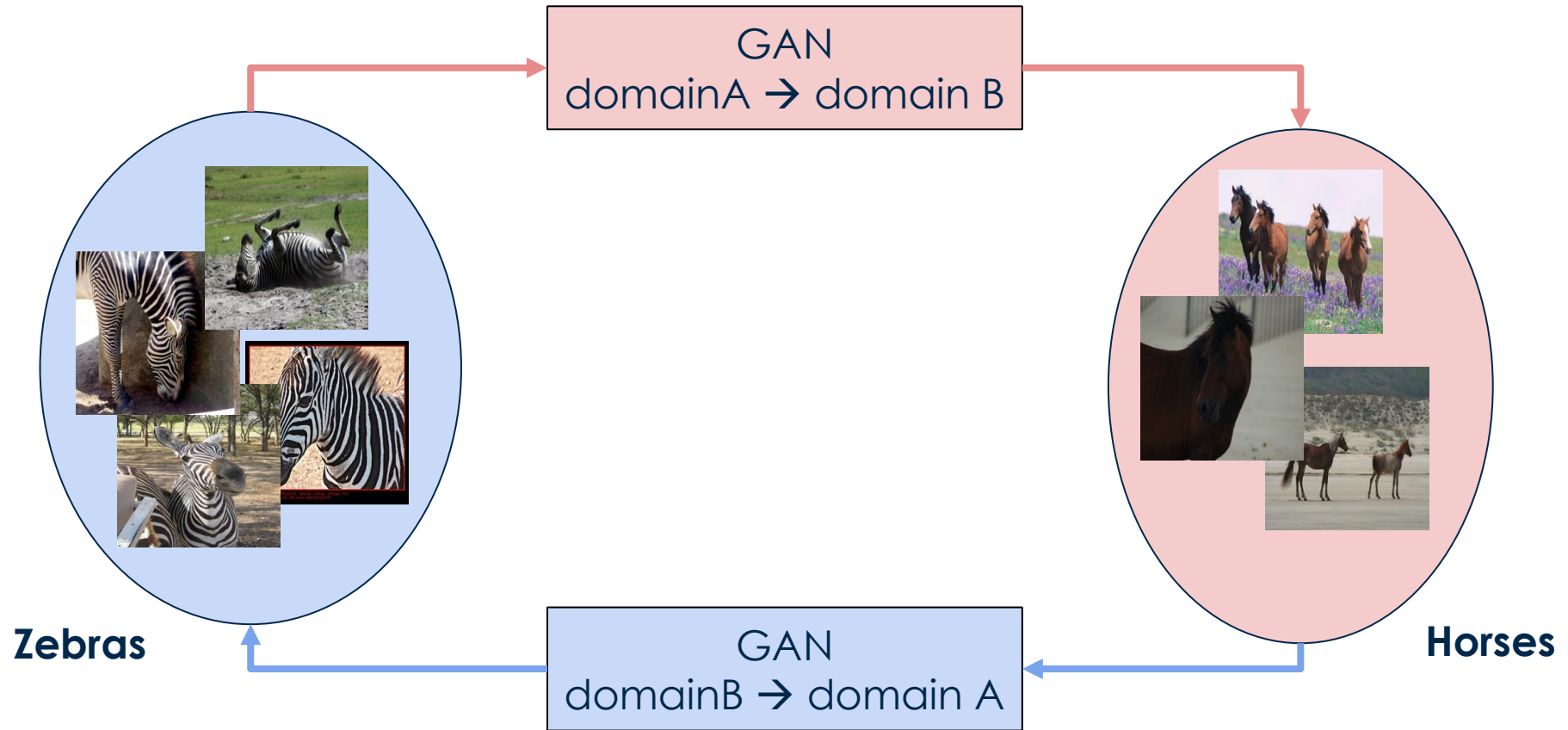
Real and fake zebra MUST match ($\rightarrow$ **cycle consistency**)



CycleGAN: two GANs

Cycle consistency (generators) $\rightarrow$ preserve contents

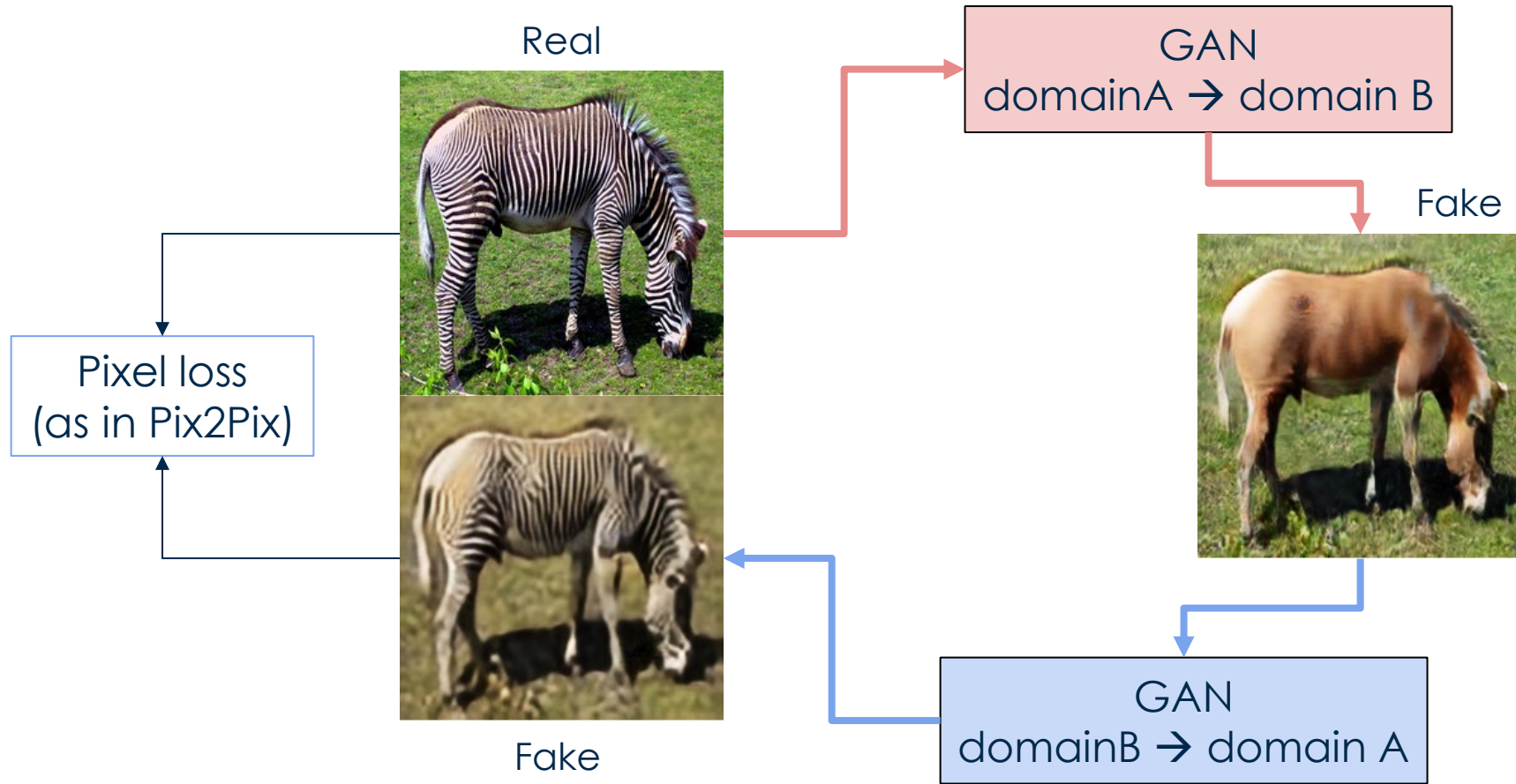
Discriminator loss $\rightarrow$ style transfer



Dataset available @ <https://github.com/togheppi/CycleGAN>

Loss: cycle consistency loss

Loss based on pixel differences for both loop directions



Loss: cycle consistency loss

Cycle consistency loss is the sum of pixel losses in both cycle directions

$$\mathcal{L}_{cycle_consistency} = \sum_{i=1}^n \left| \begin{array}{c} \text{Zebra} \\ \text{Image} \end{array} - \begin{array}{c} \text{Zebra} \\ \text{Image} \end{array} \right| + \sum_{i=1}^n \left| \begin{array}{c} \text{Horse} \\ \text{Image} \end{array} - \begin{array}{c} \text{Horse} \\ \text{Image} \end{array} \right|$$

Zebra → Horse → Zebra

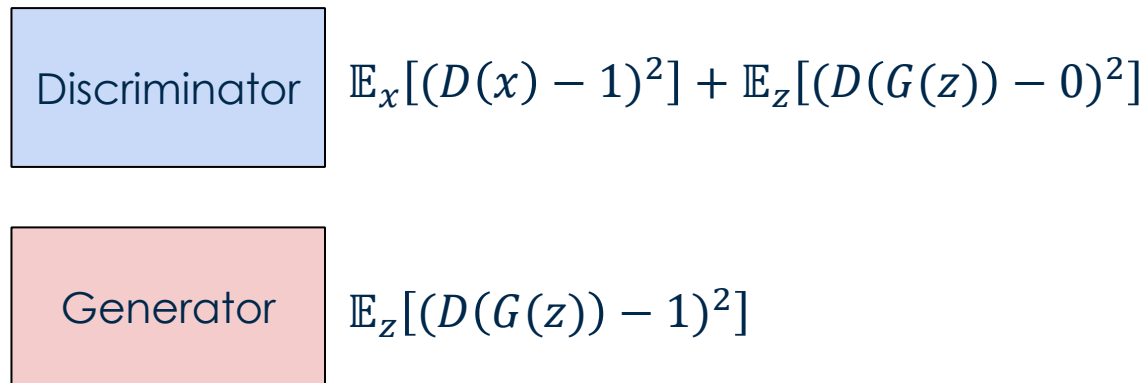
Horse → Zebra → Horse

Sum to adversarial loss using a weight

Adversarial loss

Uses least square loss (helps with vanishing gradient and mode collapse)

- › Output is patch-based (i.e., consider a sum over all patches)

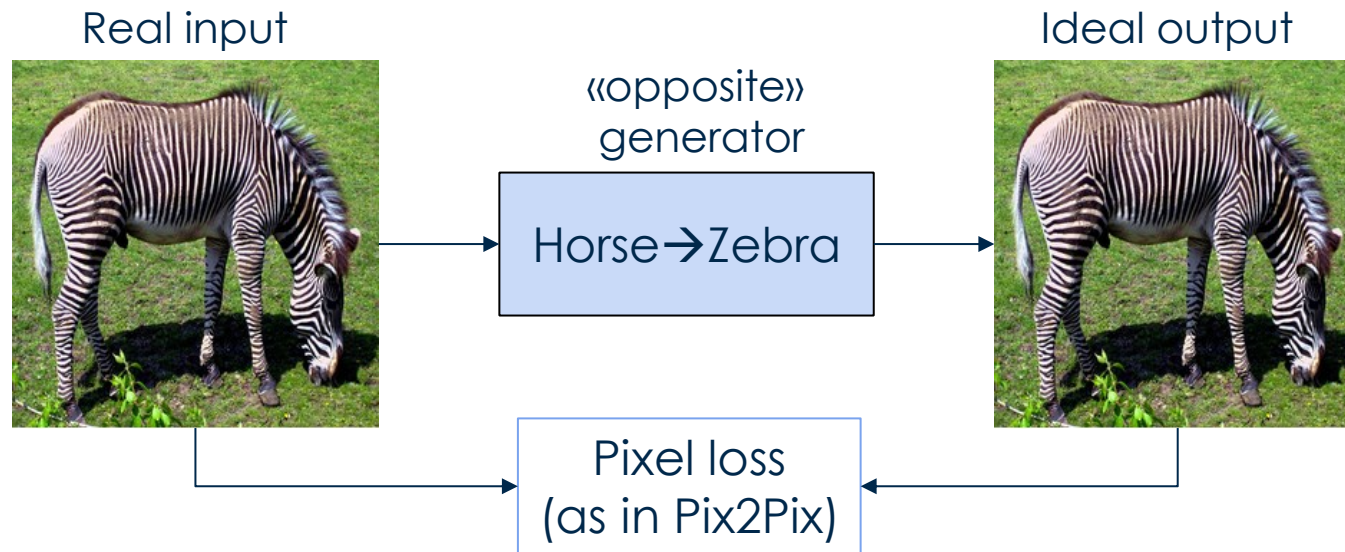


Identity loss

Extra loss term (optional) to help color preservation in the output

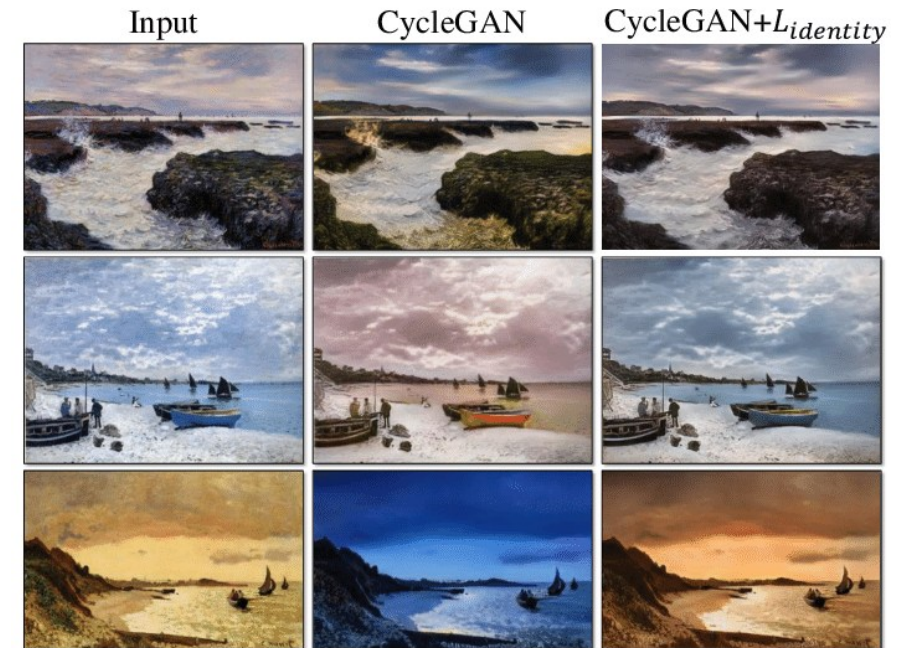
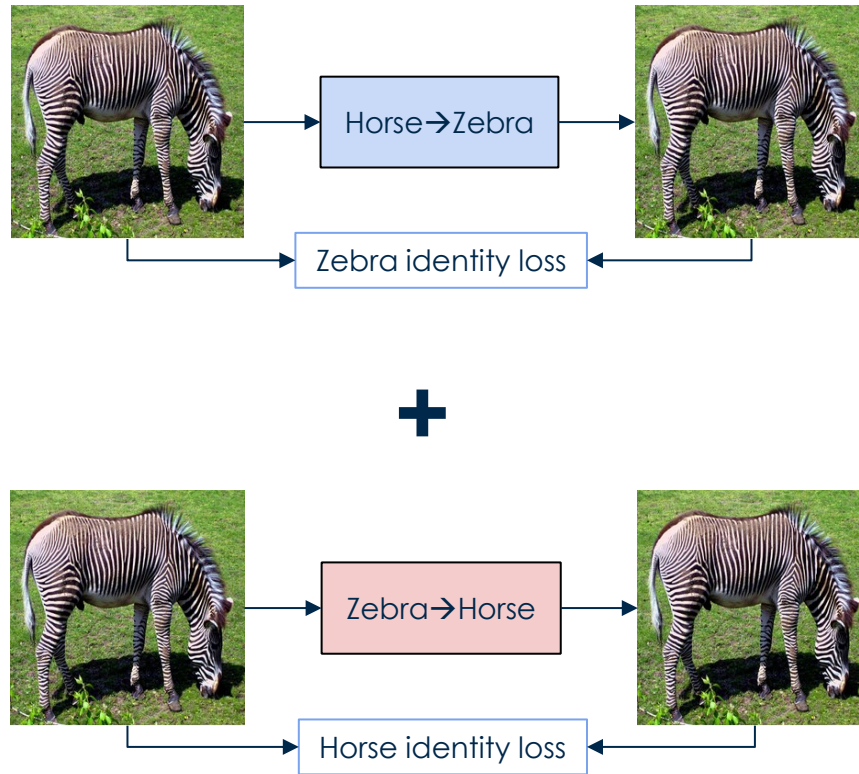
Enforces identity at pixel level (discourages color distortion)

- › A sample from domain A processed by the OPPOSITE generator (domain B $\rightarrow$ domain A) must remain unchanged



Identity loss

Two identity losses (one for each domain)



CycleGAN loss

$$\mathcal{L}_{cycleGAN} = \mathcal{L}_{adversarial} + \lambda_1 \cdot \mathcal{L}_{cycle_consistency} + \lambda_2 \cdot \mathcal{L}_{identity}$$

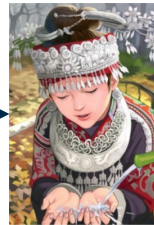
CycleGAN loss	=	L2 adversarial loss $A \rightarrow B$	+	Cycle consistency loss $A \rightarrow B \rightarrow A$	+	Identity loss $B \rightarrow A$
	+	L2 adversarial loss $B \rightarrow A$	+	Cycle consistency loss $B \rightarrow A \rightarrow B$	+	Identity loss $A \rightarrow B$

Image super-resolution

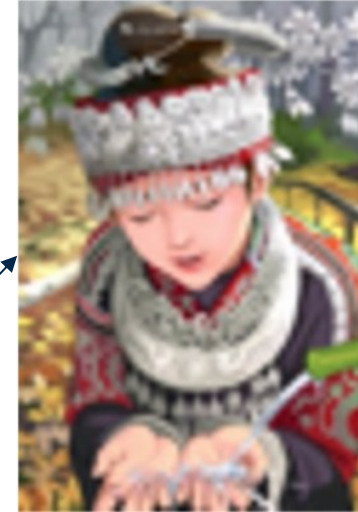
Increase image resolution creating
«new» pixel contents



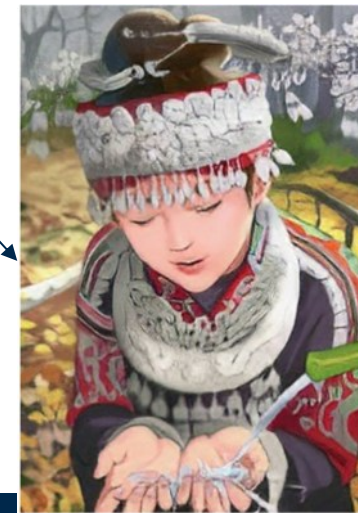
Original



Downsampled



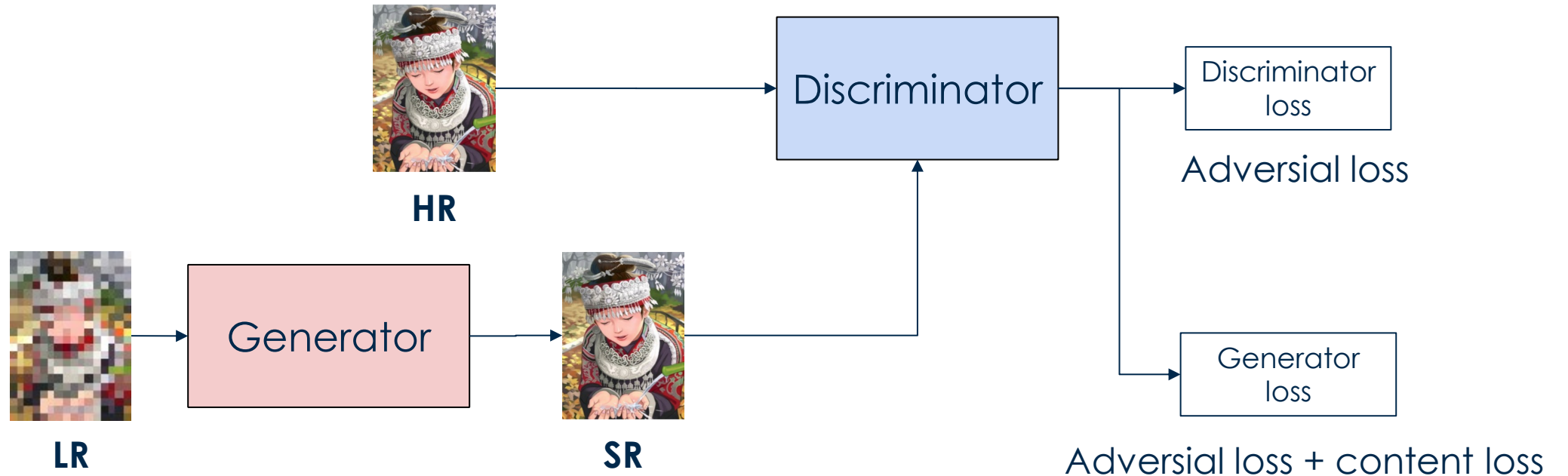
Bicubic
upsampling



GAN based
upsampling

SRResNet

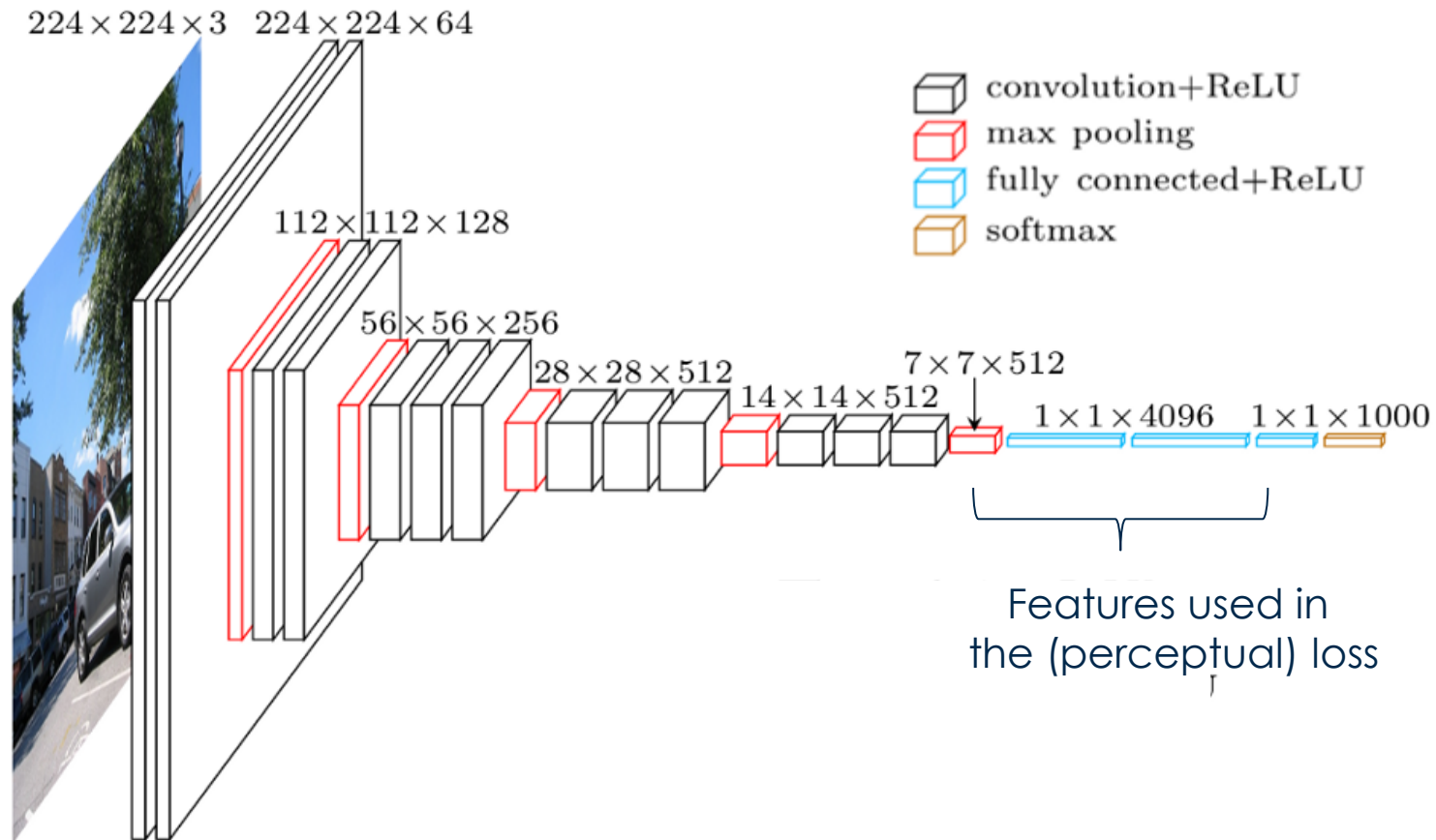
- DCGAN like architecture.
- During training high resolution images (HR) are downsampled to low-resolution (LR) and fed to a generator that creates a super-resolution (SR) of LR



SR loss

The final loss function is based on the VGG network

- › It is reknown as a good feature extractor



Generator loss

Adversarial loss + content loss

- › Adversarial loss is the standard log loss
- › Content loss aims at minimizing the perceptual loss between HR and SR $\rightarrow$ MSE between VGG-19 features (from FC layers)
- › VGG-19 features encode structural information

